

From Classical Single-Marker QTL Mapping to Multi-Marker Shrinkage: Analysis and Simulation Using Cotton MAGIC Genotypes

Shiyan Miao

2026-03-24

Table of contents

0.1	Required packages	1
0.2	File paths	1
0.3	Import data	2
0.4	Inspect dimensions	2
0.5	Add marker names to the genotype matrix	2
0.6	Construct line and season identifiers for phenotype data	3
0.7	Merge phenotype and genotype	4
1	Descriptive Analysis	4
1.1	Trait names	4
1.2	Summary statistics	4
1.3	Season-specific summaries	4
1.4	Trait distributions	5
1.5	Trait boxplots by season	7
1.6	Correlation among traits	10
1.6.1		38
1.6.2		43

0.1 Required packages

```
library(tidyverse)
library(data.table)
library(glmnet)
library(broom)
library(janitor)
library(patchwork)
library(Matrix)
library(knitr)
library(scales)
library(irlba)
```

0.2 File paths

```
geno_file <- "Genotype_data.txt"
map_file <- "Genetic_map.txt"
pheno_file <- "Phenotype_data.txt"
```

0.3 Import data

```
geno_raw <- fread(geno_file, header = FALSE)
map_df <- fread(map_file)
pheno_raw <- fread(pheno_file)
```

0.4 Inspect dimensions

```
dim(geno_raw)
```

```
[1] 256 6523
```

```
dim(map_df)
```

```
[1] 6523 3
```

```
dim(pheno_raw)
```

```
[1] 768 4
```

```
head(map_df)
```

	SNP_ID	Chromosome	Position_bp
	<char>	<char>	<int>
1:	Marker-1	A01	373964
2:	Marker-2	A01	432953
3:	Marker-3	A01	506435
4:	Marker-4	A01	514057
5:	Marker-5	A01	514827
6:	Marker-6	A01	528238

```
head(pheno_raw)
```

	Lint_percentage	Seed_index	Lint_index	Seed_oil_content
	<num>	<num>	<num>	<num>
1:	37.5	11.3	8.1	18.2
2:	38.9	9.6	7.2	19.8
3:	41.7	8.5	7.1	19.8
4:	40.9	9.0	7.3	19.8
5:	38.4	10.5	7.9	20.2
6:	36.6	10.2	6.8	19.2

This block establishes the analytical dataset. The report confirms a **256 × 6523** genotype matrix, a **6523 × 3** marker map, and a **768 × 4** phenotype file. On that basis, it infers that the phenotype file represents **256 lines observed across 3 stacked seasons/environments**, reconstructs Line and Season, and merges phenotype with genotype into `analysis_df` with **768 rows and 6530 columns**.

0.5 Add marker names to the genotype matrix

The genotype file has no column names. We use the SNP identifiers from the genetic map.

```
stopifnot(ncol(geno_raw) == nrow(map_df))
```

```
geno_df <- as.data.frame(geno_raw)
colnames(geno_df) <- map_df$SNP_ID
```

```
geno_df$Line <- 1:nrow(geno_df)

geno_df[1:5, 1:8]
```

	Marker-1	Marker-2	Marker-3	Marker-4	Marker-5	Marker-6	Marker-7	Marker-8
1	1	-1	1.000	-1	1	-1	-1	-1
2	1	-1	0.305	-1	0	-1	-1	0
3	0	1	-0.710	0	-1	1	0	1
4	1	0	1.000	-1	0	-1	-1	-1
5	1	-1	0.000	-1	1	-1	-1	-1

0.6 Construct line and season identifiers for phenotype data

The phenotype file contains 768 rows, while the genotype matrix contains 256 lines. This strongly suggests that phenotype observations correspond to three stacked seasons or environments for the same 256 lines.

```
n_lines <- nrow(geno_df)
n_pheno <- nrow(pheno_raw)

n_pheno / n_lines
```

```
[1] 3
```

If the phenotype observations are indeed stacked by season, then each block of 256 rows corresponds to one season.

```
pheno_df <- pheno_raw %>%
  mutate(
    RowID = row_number(),
    Season = case_when(
      RowID <= n_lines ~ "Season_1",
      RowID <= 2 * n_lines ~ "Season_2",
      TRUE ~ "Season_3"
    ),
    Line = rep(1:n_lines, times = n_pheno / n_lines)
  ) %>%
  relocate(Line, Season, RowID)

head(pheno_df)
```

	Line	Season	RowID	Lint_percentage	Seed_index	Lint_index	Seed_oil_content
	<int>	<char>	<int>	<num>	<num>	<num>	<num>
1:	1	Season_1	1	37.5	11.3	8.1	18.2
2:	2	Season_1	2	38.9	9.6	7.2	19.8
3:	3	Season_1	3	41.7	8.5	7.1	19.8
4:	4	Season_1	4	40.9	9.0	7.3	19.8
5:	5	Season_1	5	38.4	10.5	7.9	20.2
6:	6	Season_1	6	36.6	10.2	6.8	19.2

```
tail(pheno_df)
```

	Line	Season	RowID	Lint_percentage	Seed_index	Lint_index	Seed_oil_content
	<int>	<char>	<int>	<num>	<num>	<num>	<num>
1:	251	Season_3	763	43.1	8.4	8.4	21.0
2:	252	Season_3	764	47.8	7.7	9.2	22.4
3:	253	Season_3	765	46.1	7.7	8.6	20.9

4:	254	Season_3	766	44.7	7.8	7.9	23.2
5:	255	Season_3	767	44.7	7.2	7.1	21.4
6:	256	Season_3	768	40.4	8.1	7.1	23.4

0.7 Merge phenotype and genotype

```
analysis_df <- pheno_df %>%
  left_join(geno_df, by = "Line")

dim(analysis_df)
```

```
[1] 768 6530
```

1 Descriptive Analysis

1.1 Trait names

```
trait_names <- c(
  "Lint_percentage",
  "Seed_index",
  "Lint_index",
  "Seed_oil_content"
)
```

1.2 Summary statistics

```
trait_summary <- analysis_df %>%
  summarise(across(all_of(trait_names), list(
    mean = ~mean(.x, na.rm = TRUE),
    sd = ~sd(.x, na.rm = TRUE),
    min = ~min(.x, na.rm = TRUE),
    max = ~max(.x, na.rm = TRUE)
  )))
```

```
trait_summary
```

	Lint_percentage_mean	Lint_percentage_sd	Lint_percentage_min		
1	42.44674	2.866789	30.7		
	Lint_percentage_max	Seed_index_mean	Seed_index_sd	Seed_index_min	
1	49.4	8.48776	0.9811289	6.3	
	Seed_index_max	Lint_index_mean	Lint_index_sd	Lint_index_min	Lint_index_max
1	12.3	7.607422	0.5488731	5.9	9.4
	Seed_oil_content_mean	Seed_oil_content_sd	Seed_oil_content_min		
1	19.7099	1.791653	15.2		
	Seed_oil_content_max				
1	26.9				

1.3 Season-specific summaries

```
season_summary <- analysis_df %>%
  group_by(Season) %>%
  summarise(across(all_of(trait_names), list(
```

```

    mean = ~mean(.x, na.rm = TRUE),
    sd = ~sd(.x, na.rm = TRUE)
  )), .groups = "drop")

season_summary

# A tibble: 3 x 9
  Season  Lint_percentage_mean Lint_percentage_sd Seed_index_mean Seed_index_sd
  <chr>      <dbl>          <dbl>          <dbl>          <dbl>
1 Season_1      39.5            1.92            9.58            0.705
2 Season_2      43.9            1.95            8.15            0.500
3 Season_3      43.9            2.03            7.73            0.499
# i 4 more variables: Lint_index_mean <dbl>, Lint_index_sd <dbl>,
#   Seed_oil_content_mean <dbl>, Seed_oil_content_sd <dbl>

```

1.4 Trait distributions

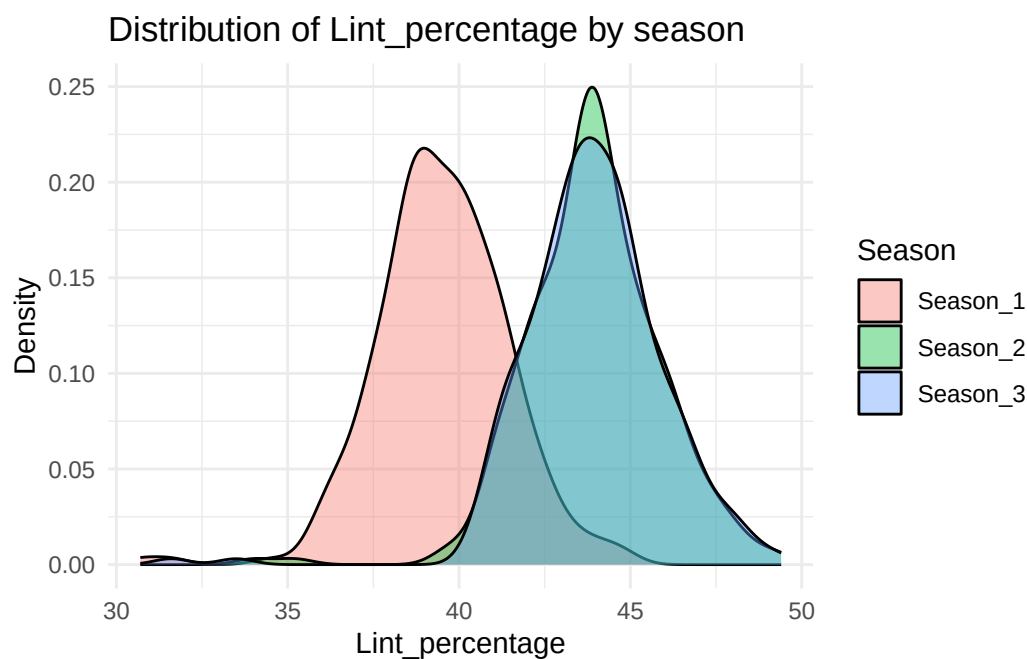
```

plot_trait_density <- function(df, trait) {
  ggplot(df, aes(x = .data[[trait]], fill = Season)) +
    geom_density(alpha = 0.4) +
    labs(
      title = paste("Distribution of", trait, "by season"),
      x = trait,
      y = "Density"
    ) +
    theme_minimal()
}

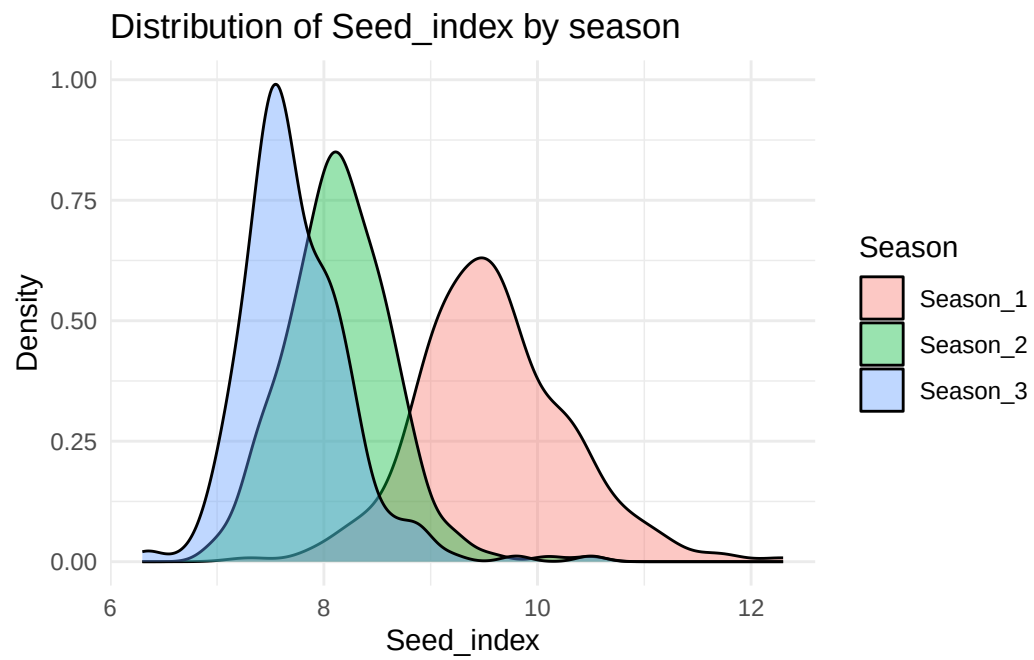
plots_density <- lapply(trait_names, function(tr) plot_trait_density(analysis_df, tr))

plots_density[[1]]

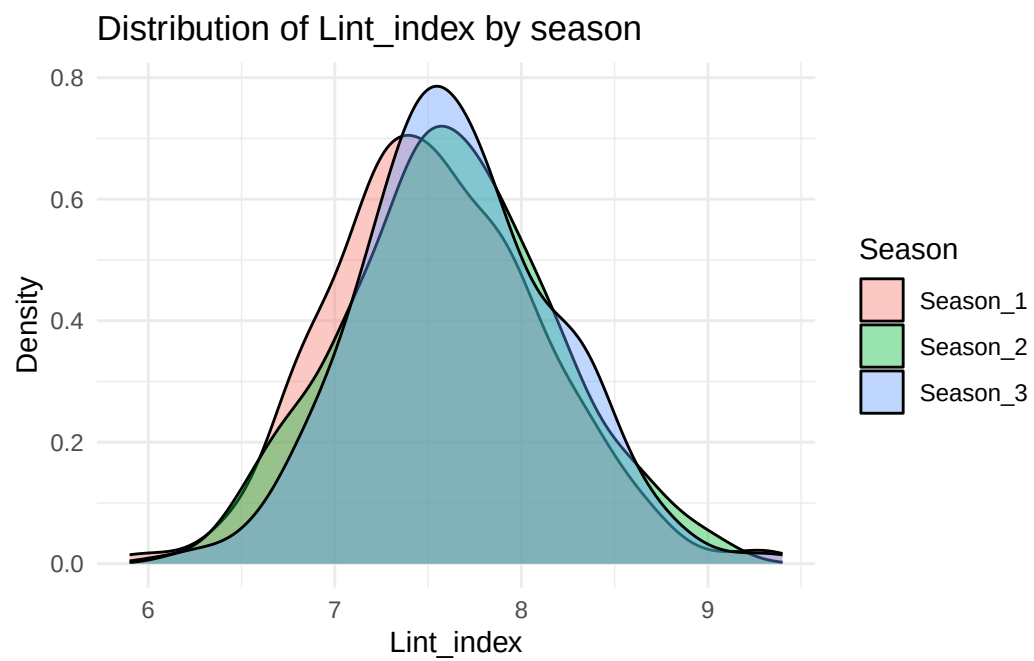
```



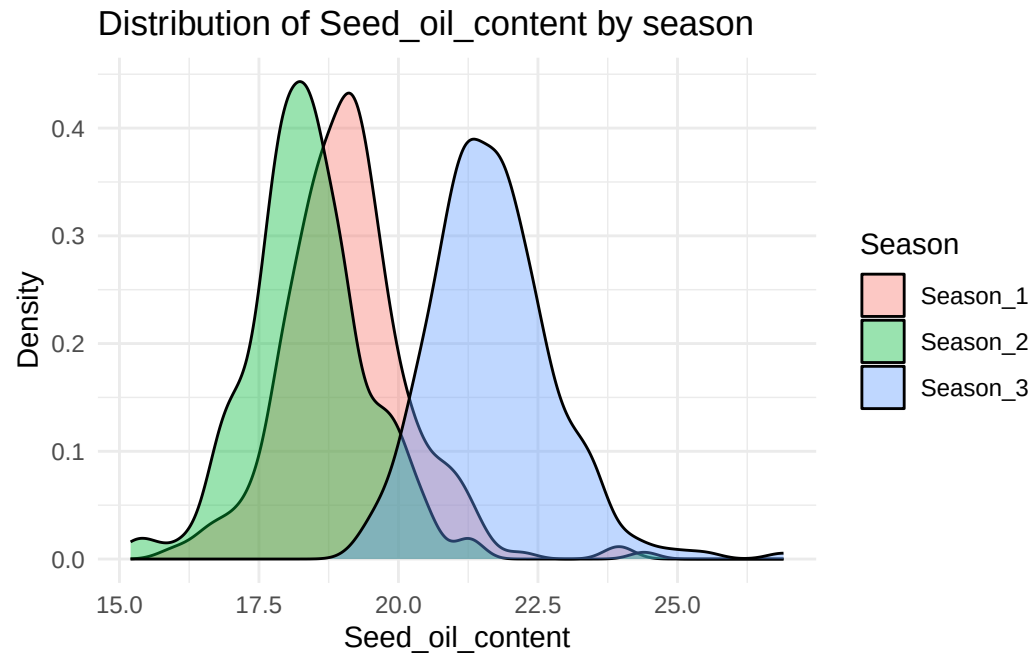
```
plots_density[[2]]
```



```
plots_density[[3]]
```



```
plots_density[[4]]
```

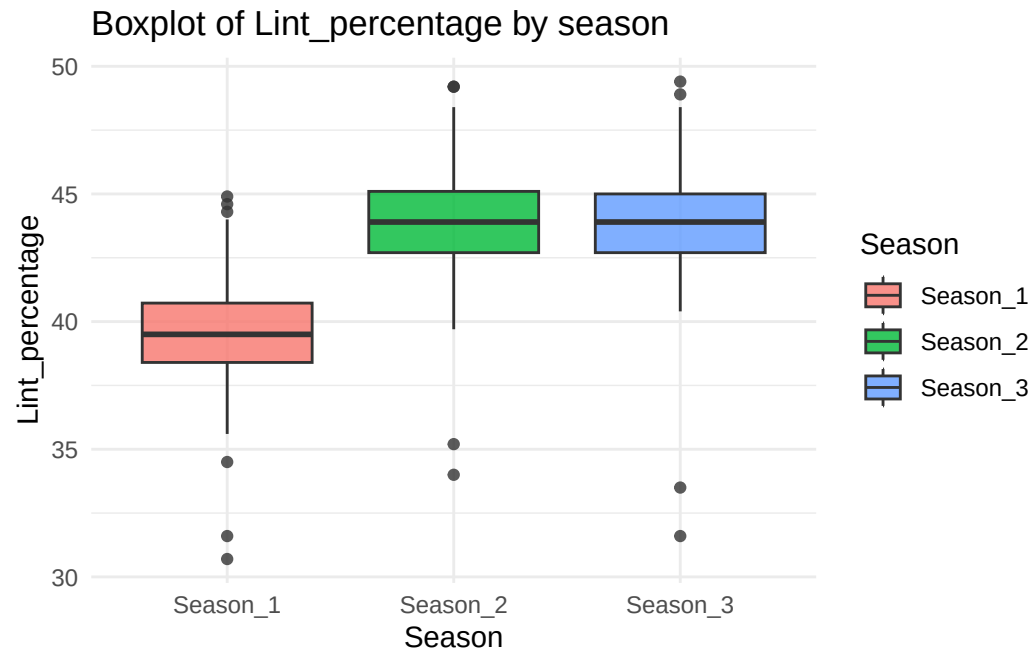


1.5 Trait boxplots by season

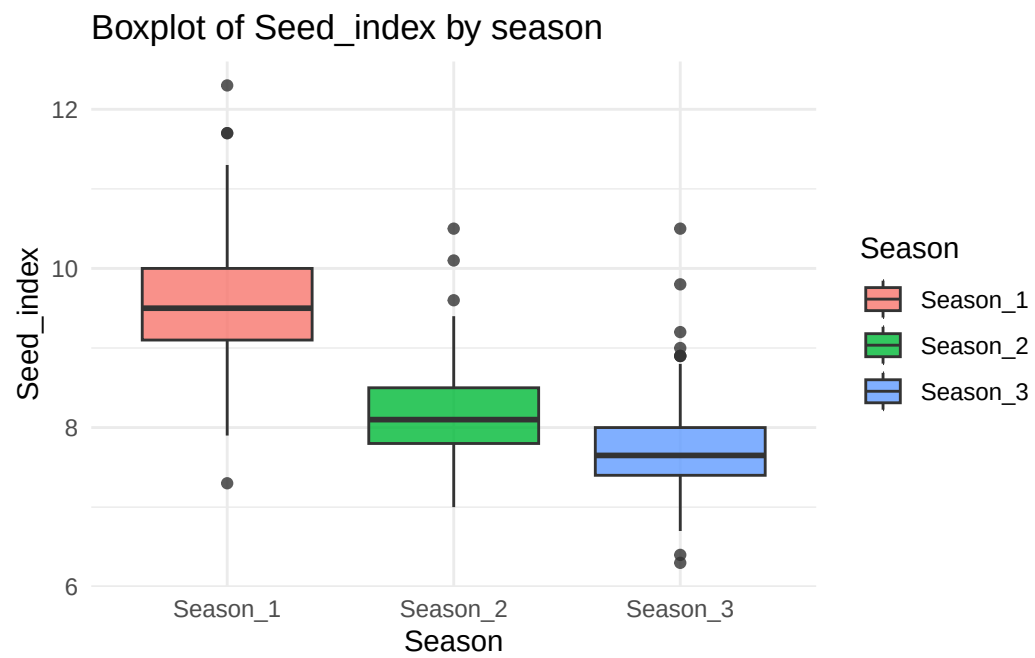
```
plot_trait_box <- function(df, trait) {
  ggplot(df, aes(x = Season, y = .data[[trait]], fill = Season)) +
    geom_boxplot(alpha = 0.8) +
    labs(
      title = paste("Boxplot of", trait, "by season"),
      x = "Season",
      y = trait
    ) +
    theme_minimal()
}

plots_box <- lapply(trait_names, function(tr) plot_trait_box(analysis_df, tr))

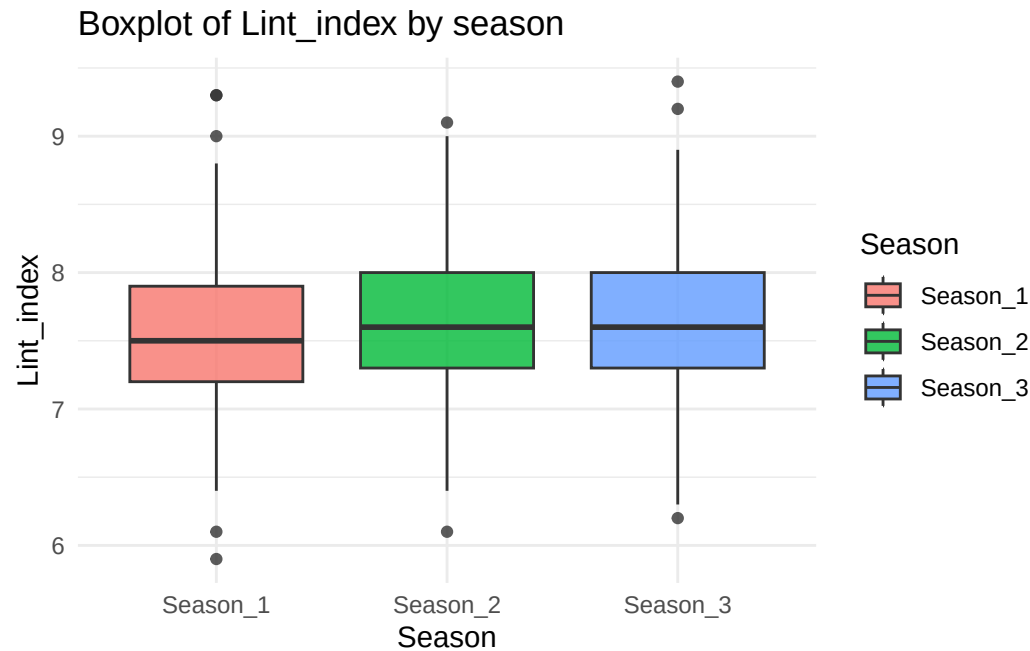
plots_box[[1]]
```



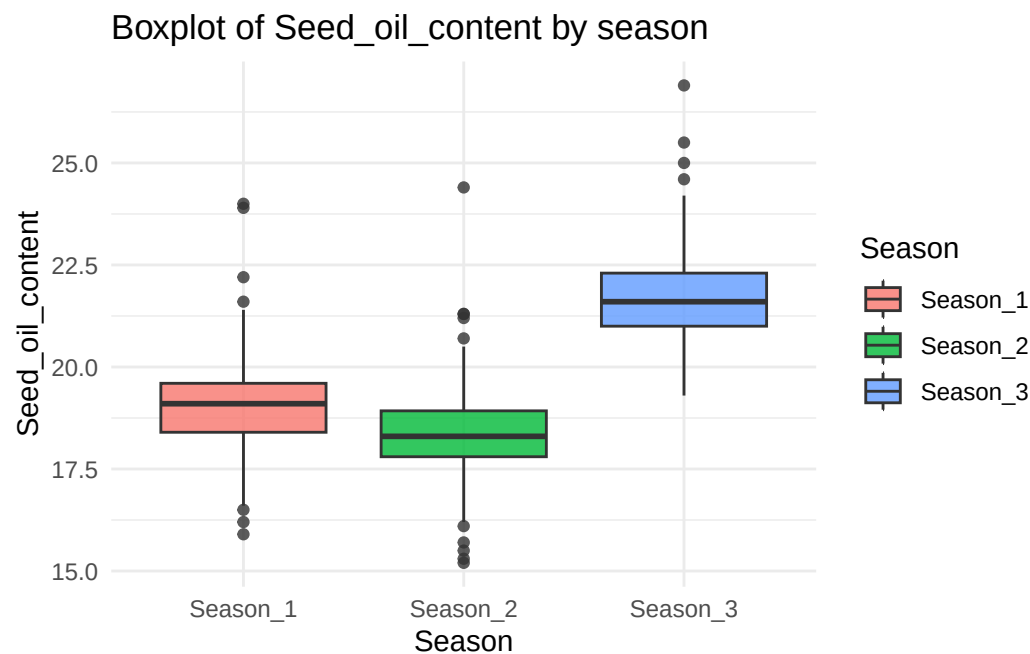
```
plots_box[[2]]
```



```
plots_box[[3]]
```

```
plots_box[[4]]
```



The report focuses on four traits: `Lint_percentage`, `Seed_index`, `Lint_index`, and `Seed_oil_content`. Across seasons, the report shows strong mean shifts: `Lint_percentage` rises from **39.5** in `Season_1` to **43.9** in `Seasons_2` and `_3`, while `Seed_index` declines from **9.58** to **8.15** and then **7.73**.

These figures show that season effects are not driven by a handful of extreme observations; they reflect **systematic distributional shifts**. They also show that substantial within-season variation remains, which is exactly what makes QTL analysis feasible.

However, the plots do not formally diagnose whether residual variance is homogeneous across seasons, nor do they account for repeated observations of the same line across seasons. So they are strong descriptive

diagnostics, but not a substitute for full model checking.

1.6 Correlation among traits

```
trait_cor <- analysis_df %>%
  select(all_of(trait_names)) %>%
  cor(use = "pairwise.complete.obs")

trait_cor
```

	Lint_percentage	Seed_index	Lint_index	Seed_oil_content
Lint_percentage	1.00000000	-0.7783457	0.35414014	0.05702825
Seed_index	-0.77834569	1.00000000	0.14117179	-0.27722812
Lint_index	0.35414014	0.1411718	1.00000000	-0.07199958
Seed_oil_content	0.05702825	-0.2772281	-0.07199958	1.00000000

The correlation matrix shows a very strong negative association between `Lint_percentage` and `Seed_index` (**-0.778**), a moderate positive association between `Lint_percentage` and `Lint_index` (**0.354**), a weaker negative association between `Seed_index` and `Seed_oil_content` (**-0.277**), and near-zero correlation between `Seed_oil_content` and `Lint_percentage` (**0.057**).

This is an important structural result. The strong `Lint_percentage`–`Seed_index` trade-off suggests that hits shared across those traits may reflect a common underlying axis rather than independent mechanisms. By contrast, `Seed_oil_content` appears much more weakly connected to the others, implying a comparatively distinct genetic dimension. In substantive terms, the correlation matrix already suggests a system with at least one major **lint–seed trade-off axis** and a more separate **oil-content axis**.

```
head(geno_raw[, 1:min(8, ncol(geno_raw))])
```

	V1	V2	V3	V4	V5	V6	V7	V8
	<int>	<int>	<num>	<int>	<int>	<int>	<int>	<int>
1:	1	-1	1.000	-1	1	-1	-1	-1
2:	1	-1	0.305	-1	0	-1	-1	0
3:	0	1	-0.710	0	-1	1	0	1
4:	1	0	1.000	-1	0	-1	-1	-1
5:	1	-1	0.000	-1	1	-1	-1	-1
6:	0	1	-1.000	0	-1	1	-1	0

```
head(map_df)
```

	SNP_ID	Chromosome	Position_bp
	<char>	<char>	<int>
1:	Marker-1	A01	373964
2:	Marker-2	A01	432953
3:	Marker-3	A01	506435
4:	Marker-4	A01	514057
5:	Marker-5	A01	514827
6:	Marker-6	A01	528238

```
head(pheno_raw)
```

	Lint_percentage	Seed_index	Lint_index	Seed_oil_content
	<num>	<num>	<num>	<num>
1:	37.5	11.3	8.1	18.2
2:	38.9	9.6	7.2	19.8
3:	41.7	8.5	7.1	19.8
4:	40.9	9.0	7.3	19.8

```

5:          38.4          10.5          7.9          20.2
6:          36.6          10.2          6.8          19.2

```

The purpose of this step is to confirm that the imported files match the expected Week-3 data structure. In the previous report, this dataset consisted of a 256-line genotype matrix, a 6523-marker map, and a 768-row phenotype file. If the imported dimensions are broadly consistent with this structure, then the current analysis can proceed on the same empirical basis as the earlier report.

```
stopifnot(ncol(geno_raw) == nrow(map_df))
```

```

geno_df <- as.data.frame(geno_raw)
colnames(geno_df) <- map_df$SNP_ID
geno_df$Line <- 1:nrow(geno_df)

```

```
knitr::kable(head(map_df))
```

SNP_ID	Chromosome	Position_bp
Marker-1	A01	373964
Marker-2	A01	432953
Marker-3	A01	506435
Marker-4	A01	514057
Marker-5	A01	514827
Marker-6	A01	528238

The raw genotype file does not contain marker names, so it cannot be meaningfully interpreted or used for PCA in its imported form. This step therefore aligns the genotype matrix with the marker map by assigning the SNP_ID values from the map file as column names. It also introduces a **Line** variable, which is necessary for matching genotype rows to reconstructed phenotype line identities.

```

n_lines <- nrow(geno_df)
n_pheno <- nrow(pheno_raw)

```

```

pheno_df <- pheno_raw %>%
  mutate(
    RowID = row_number(),
    Season = case_when(
      RowID <= n_lines ~ "Season_1",
      RowID <= 2 * n_lines ~ "Season_2",
      TRUE ~ "Season_3"
    ),
    Line = rep(1:n_lines, times = n_pheno / n_lines)
  ) %>%
  relocate(Line, Season, RowID)

```

```
head(pheno_df)
```

	Line	Season	RowID	Lint_percentage	Seed_index	Lint_index	Seed_oil_content
	<int>	<char>	<int>	<num>	<num>	<num>	<num>
1:	1	Season_1	1	37.5	11.3	8.1	18.2
2:	2	Season_1	2	38.9	9.6	7.2	19.8
3:	3	Season_1	3	41.7	8.5	7.1	19.8
4:	4	Season_1	4	40.9	9.0	7.3	19.8
5:	5	Season_1	5	38.4	10.5	7.9	20.2
6:	6	Season_1	6	36.6	10.2	6.8	19.2

This step reconstructs the phenotype structure by assigning both a `Line` identifier and a `Season` label, thereby creating a bridge between the phenotype and genotype data.

```
analysis_df <- pheno_df %>%
  left_join(geno_df, by = "Line")

merge_tbl <- tibble(
  object = c("analysis_df"),
  nrow = nrow(analysis_df),
  ncol = ncol(analysis_df)
)

knitr::kable(merge_tbl)
```

object	nrow	ncol
analysis_df	768	6530

This step merges phenotype and genotype information by line. Although PCA itself only requires the genotype matrix, integrating phenotype into the broader analysis object is useful for two reasons: it allows later interpretation of the PCs in relation to the traits, and it enables phenotype-based colouring or annotation in PCA visualisations.

```
X <- geno_df %>%
  select(-Line) %>%
  as.matrix()

mode(X) <- "numeric"

for (j in seq_len(ncol(X))) {
  miss_idx <- is.na(X[, j])
  if (any(miss_idx)) {
    X[miss_idx, j] <- mean(X[, j], na.rm = TRUE)
  }
}
```

PCA is applied only to the genotype matrix, so this section extracts the marker matrix from `geno_df`, removes the `Line` identifier, and converts the result into a numeric matrix `X`. Any missing values are mean-imputed on a marker-by-marker basis for PCA diagnostics. This imputation is used only to enable dimensional reduction and does not imply that the same strategy must be used in later modelling.

```
set.seed(123)

pca_fit <- prcomp_irlba(
  x = scale(X, center = TRUE, scale = TRUE),
  n = 20,
  center = FALSE,
  scale. = FALSE
)

var_explained <- (pca_fit$sdev^2) / sum(pca_fit$sdev^2)
cum_var_explained <- cumsum(var_explained)

pca_var_df <- tibble(
  PC = 1:length(var_explained),
```

```

Variance_Explained = var_explained,
Cumulative = cum_var_explained
)

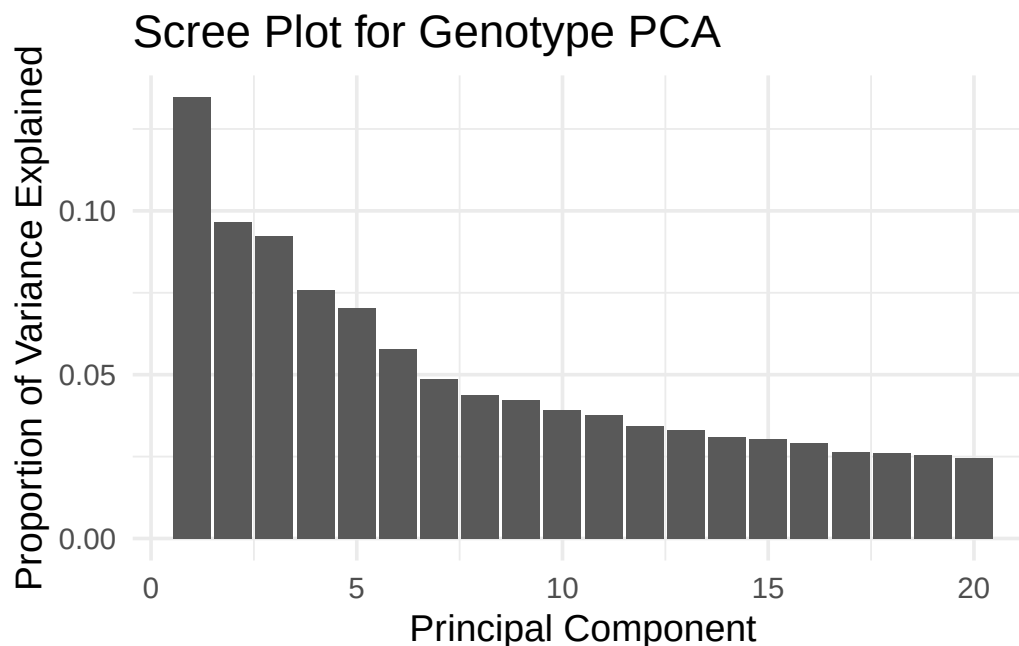
```

PCA is now applied to the genotype matrix using `prcomp_irlba()`, which is more efficient than standard PCA for larger matrices. The purpose here is not direct prediction, but structural diagnosis: to determine whether the genotype matrix contains clear lower-dimensional variation patterns, and to assess whether these patterns may matter for later multiple-marker analysis.

```

ggplot(pca_var_df, aes(x = PC, y = Variance_Explained)) +
  geom_col() +
  labs(
    title = "Scree Plot for Genotype PCA",
    x = "Principal Component",
    y = "Proportion of Variance Explained"
  ) +
  theme_minimal(base_size = 14)

```



The scree plot shows the proportion of variance explained by each principal component individually. If the first few PCs explain a substantial share of the variance, this suggests that the genotype matrix contains strong low-dimensional structure. If variance is spread thinly across many components, then the data are better characterised as highly dispersed and structurally complex.

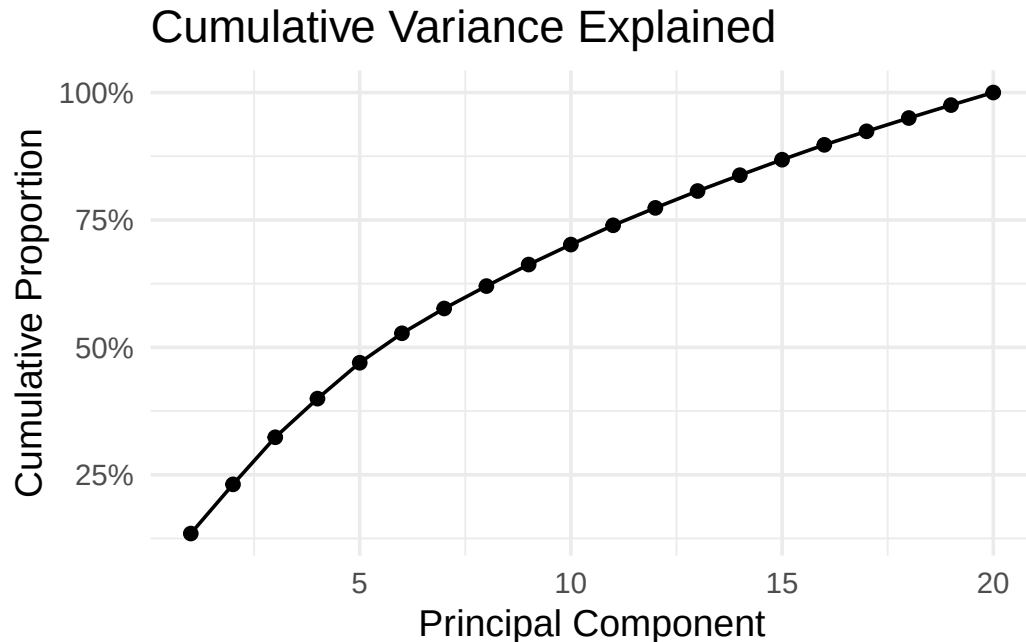
The scree plot indicates that no single principal component dominates the genotype variation; even the leading PCs explain only modest proportions of variance. The cumulative variance plot shows that multiple PCs are needed to capture a substantial share of total structure. That implies a **diffuse, multi-dimensional genotype structure** rather than a single overwhelming stratification axis.

```

ggplot(pca_var_df, aes(x = PC, y = Cumulative)) +
  geom_line() +
  geom_point() +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  labs(
    title = "Cumulative Variance Explained",

```

```
x = "Principal Component",
y = "Cumulative Proportion"
) +
theme_minimal(base_size = 14)
```



The cumulative variance plot shows how much total genotype variation is captured as more principal components are included. This is useful for evaluating whether the first 5, 10, or 20 PCs provide an adequate summary of the overall marker structure.

```
pca_scores <- as_tibble(pca_fit$x[, 1:5]) %>%
  mutate(Line = 1:n())

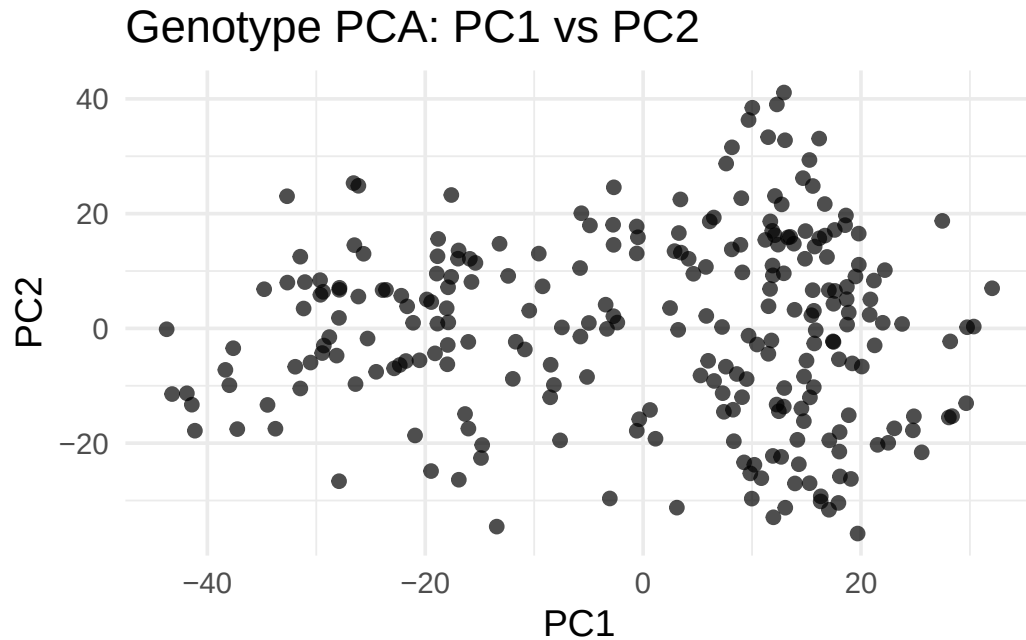
trait_line_mean <- pheno_df %>%
  group_by(Line) %>%
  summarise(
    Lint_percentage = mean(Lint_percentage, na.rm = TRUE),
    Seed_index = mean(Seed_index, na.rm = TRUE),
    Lint_index = mean(Lint_index, na.rm = TRUE),
    Seed_oil_content = mean(Seed_oil_content, na.rm = TRUE),
    .groups = "drop"
  )

pca_scores <- pca_scores %>%
  left_join(trait_line_mean, by = "Line")
```

This section extracts the PCA scores for each line on the first few principal components. It then merges in line-level mean trait values, averaged across seasons, in order to support later comparison between the genetic PCs and phenotype patterns.

```
ggplot(pca_scores, aes(x = PC1, y = PC2)) +
  geom_point(alpha = 0.7) +
  labs(
    title = "Genotype PCA: PC1 vs PC2",
```

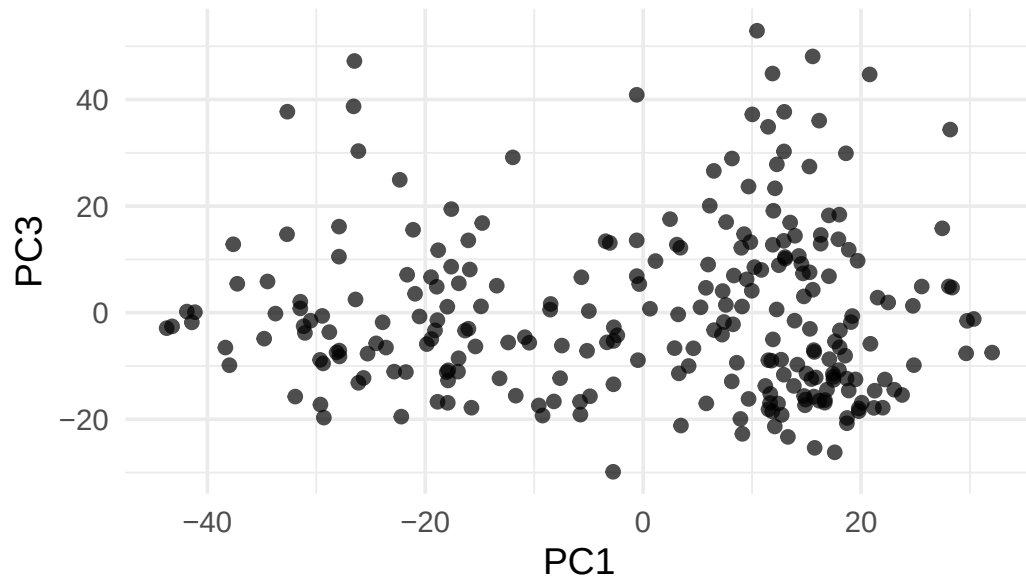
```
x = "PC1",
y = "PC2"
) +
theme_minimal(base_size = 14)
```



PC1 and PC2 typically capture the dominant axes of variation in the genotype matrix. This plot is therefore used to inspect whether the 256 lines show visible clustering, gradients, or broad separation in the first two principal components.

```
ggplot(pca_scores, aes(x = PC1, y = PC3)) +
  geom_point(alpha = 0.7) +
  labs(
    title = "Genotype PCA: PC1 vs PC3",
    x = "PC1",
    y = "PC3"
  ) +
  theme_minimal(base_size = 14)
```

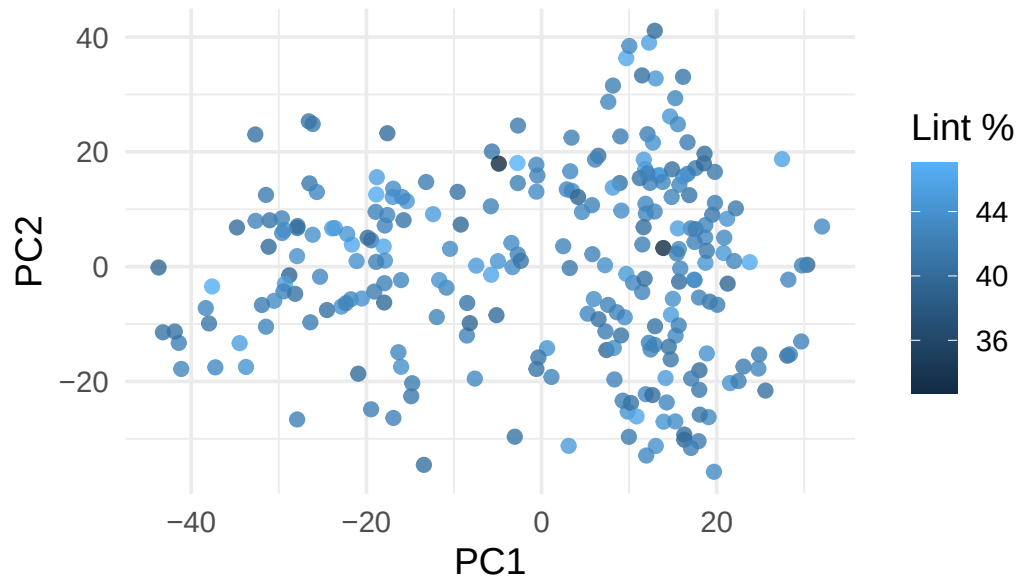
Genotype PCA: PC1 vs PC3



It is often useful to inspect PC1 against PC3 as well, because PC3 may capture variation that is not visible in PC2. This provides a broader picture of whether there are additional structural patterns in the genotype matrix.

```
ggplot(pca_scores, aes(x = PC1, y = PC2, colour = Lint_percentage)) +  
  geom_point(size = 2, alpha = 0.8) +  
  labs(  
    title = "PC1 vs PC2 coloured by average lint percentage",  
    x = "PC1",  
    y = "PC2",  
    colour = "Lint %"  
  ) +  
  theme_minimal(base_size = 14)
```


PC1 vs PC2 coloured by average lint percenta



This figure colours the PCA plot by the line-level mean of lint percentage. The purpose is not to claim that the PCs directly predict the trait, but to visually inspect whether phenotype variation appears to align with the major genomic axes.

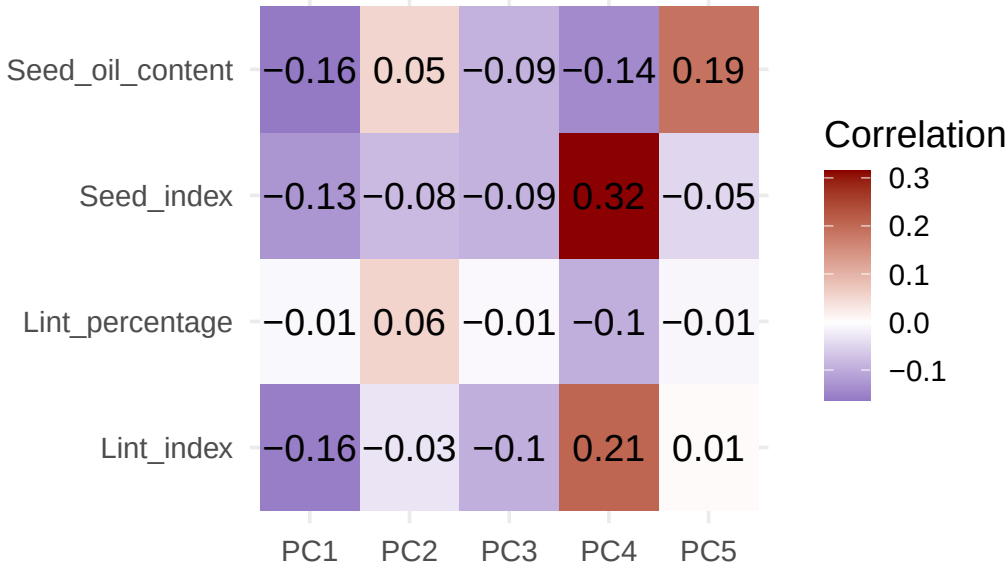
```
pc_trait_df <- pca_scores %>%
  select(Line, PC1, PC2, PC3, PC4, PC5, Lint_percentage, Seed_index, Lint_index, Seed_oil_content)

cor_mat <- pc_trait_df %>%
  select(-Line) %>%
  cor(use = "pairwise.complete.obs")

cor_df <- as.data.frame(cor_mat) %>%
  rownames_to_column("Var1") %>%
  pivot_longer(-Var1, names_to = "Var2", values_to = "Correlation") %>%
  filter(
    Var1 %in% c("PC1", "PC2", "PC3", "PC4", "PC5"),
    Var2 %in% c("Lint_percentage", "Seed_index", "Lint_index", "Seed_oil_content")
  )

ggplot(cor_df, aes(x = Var1, y = Var2, fill = Correlation)) +
  geom_tile() +
  geom_text(aes(label = round(Correlation, 2)), size = 5) +
  scale_fill_gradient2(low = "darkblue", mid = "white", high = "darkred", midpoint = 0) +
  labs(
    title = "Correlation between top PCs and averaged traits",
    x = NULL,
    y = NULL
  ) +
  theme_minimal(base_size = 14)
```

Correlation between top PCs and average



The PC1–PC2 and PC1–PC3 scatterplots show that the 256 lines are not randomly dispersed; there is visible broad separation and gradient-like structure, especially along PC1, but not sharply isolated clusters. The colour-coded `Lint_percentage` PCA plot does not show a clean phenotypic partition along the first two genotype axes. The heatmap supports that interpretation: PC–trait correlations are generally modest, with the strongest values being **PC4–Lint_index = 0.32**, **PC5–Lint_index = 0.21**, and **PC5–Seed_oil_content = 0.19**.

Population structure is present and worth controlling, but it does not by itself explain the phenotype patterns. This justifies the use of top PCs as broad structural covariates rather than as primary trait predictors.

The limitations are also clear. Missing genotypes were mean-imputed marker by marker for PCA, which is acceptable for exploratory dimensional diagnosis but may change tail variation. The choice to include only **PC1–PC5** is sensible, but still heuristic. Most importantly, because the scatterplots suggest continuous relatedness rather than neat discrete clusters, PC adjustment alone may be insufficient; a kinship-aware mixed modelling strategy would be stronger.

The baseline single-marker model is:

$$y_{is} = \alpha + \beta_j G_{ij} + \gamma_s \text{Season}_s + \varepsilon_{is}$$

where y_{is} is the phenotype for line i in season s , G_{ij} is the genotype of marker j for line i , and `Season` is included as a control.

The PC-adjusted single-marker model is:

$$y_{is} = \alpha + \beta_j G_{ij} + \gamma_s \text{Season}_s + \sum_{k=1}^5 \delta_k PC_{ik} + \varepsilon_{is}$$

where the first five genotype principal components are included as additional covariates to account for broad population structure.

```

# Merge top 5 genotype PCs back into analysis_df

pc_line_df <- pca_scores %>%
  dplyr::select(Line, PC1, PC2, PC3, PC4, PC5)

analysis_df <- analysis_df %>%
  dplyr::left_join(pc_line_df, by = "Line")

analysis_df$Season <- factor(analysis_df$Season)
marker_names <- colnames(geno_df)[colnames(geno_df) != "Line"]

# Prepare marker matrix
prep_marker_matrix <- function(df, marker_names) {
  X <- as.matrix(df[, marker_names, drop = FALSE])
  mode(X) <- "numeric"

  if (anyNA(X)) {
    col_means <- colMeans(X, na.rm = TRUE)
    idx <- which(is.na(X), arr.ind = TRUE)
    X[idx] <- col_means[idx[, 2]]
  }

  X
}

# Control Seasons and PCs
fast_single_marker_scan_cov <- function(df, trait, marker_names, covariates = "Season") {
  df <- as.data.frame(df)

  needed_cols <- unique(c(trait, covariates, marker_names))
  df <- df[, needed_cols, drop = FALSE]

  keep <- complete.cases(df[, c(trait, covariates), drop = FALSE])
  df <- df[keep, , drop = FALSE]

  y <- df[[trait]]
  X <- prep_marker_matrix(df, marker_names)

  cov_formula <- as.formula(
    paste("~", paste(covariates, collapse = " + "))
  )
  C <- model.matrix(cov_formula, data = df)
  qrC <- qr(C)

  # Residualise phenotype and marker matrix on covariates
  y_res <- qr.resid(qrC, y)
  X_res <- qr.resid(qrC, X)

  Sxx <- colSums(X_res^2)
  Sxy <- as.vector(crossprod(X_res, y_res))
  Syy <- sum(y_res^2)

  df_resid <- nrow(df) - qrC$rank - 1
  valid <- Sxx > 1e-12

```

```

beta <- rep(NA_real_, length(marker_names))
rss <- rep(NA_real_, length(marker_names))
se <- rep(NA_real_, length(marker_names))
tval <- rep(NA_real_, length(marker_names))
pval <- rep(NA_real_, length(marker_names))

beta[valid] <- Sxy[valid] / Sxx[valid]
rss[valid] <- pmax(Syy - (Sxy[valid]^2 / Sxx[valid]), 0)
se[valid] <- sqrt((rss[valid] / df_resid) / Sxx[valid])
tval[valid] <- beta[valid] / se[valid]
pval[valid] <- 2 * pt(abs(tval[valid]), df = df_resid, lower.tail = FALSE)

tibble::tibble(
  trait = trait,
  SNP_ID = marker_names,
  estimate = beta,
  std.error = se,
  statistic = tval,
  p.value = pval,
  covariates = paste(covariates, collapse = " + "),
  df_resid = df_resid
)
}

```

```

# Covariate sets

covars_season_only <- c("Season")
covars_season_pc <- c("Season", "PC1", "PC2", "PC3", "PC4", "PC5")

# Run scans for all traits

scan_season_only <- lapply(trait_names, function(tr) {
  fast_single_marker_scan_cov(
    df = analysis_df,
    trait = tr,
    marker_names = marker_names,
    covariates = covars_season_only
  )
}) %>%
  dplyr::bind_rows() %>%
  dplyr::left_join(map_df, by = "SNP_ID") %>%
  dplyr::mutate(log10p = -log10(p.value))

scan_season_pc <- lapply(trait_names, function(tr) {
  fast_single_marker_scan_cov(
    df = analysis_df,
    trait = tr,
    marker_names = marker_names,
    covariates = covars_season_pc
  )
}) %>%
  dplyr::bind_rows() %>%
  dplyr::left_join(map_df, by = "SNP_ID") %>%

```

```

dplyr::mutate(log10p = -log10(p.value))

# Multiple-testing correction-
add_multiple_testing <- function(scan_df, alpha = 0.05) {
  scan_df %>%
    dplyr::group_by(trait, covariates) %>%
    dplyr::arrange(p.value, .by_group = TRUE) %>%
    dplyr::mutate(
      p_adj_bh = p.adjust(p.value, method = "BH"),
      p_adj_bonf = p.adjust(p.value, method = "bonferroni"),
      sig_raw = p.value < alpha,
      sig_bh = p_adj_bh < alpha,
      sig_bonf = p_adj_bonf < alpha
    ) %>%
    dplyr::ungroup()
}

scan_season_only_adj <- add_multiple_testing(scan_season_only)
scan_season_pc_adj <- add_multiple_testing(scan_season_pc)

sig_summary <- dplyr::bind_rows(
  scan_season_only_adj %>% dplyr::mutate(model = "Season only"),
  scan_season_pc_adj %>% dplyr::mutate(model = "Season + PC1-5")
) %>%
  dplyr::group_by(model, trait) %>%
  dplyr::summarise(
    n_raw = sum(sig_raw, na.rm = TRUE),
    n_bh = sum(sig_bh, na.rm = TRUE),
    n_bonf = sum(sig_bonf, na.rm = TRUE),
    .groups = "drop"
  )

knitr::kable(sig_summary, digits = 3)

```

model	trait	n_raw	n_bh	n_bonf
Season + PC1-5	Lint_index	2458	1711	233
Season + PC1-5	Lint_percentage	2407	1740	248
Season + PC1-5	Seed_index	2051	1159	104
Season + PC1-5	Seed_oil_content	1832	942	100
Season only	Lint_index	2588	1701	347
Season only	Lint_percentage	1892	1160	191
Season only	Seed_index	2676	2121	286
Season only	Seed_oil_content	2237	1389	273

```

trait_to_plot <- "Lint_percentage"

top_markers_compare <- scan_season_only_adj %>%
  dplyr::filter(trait == trait_to_plot) %>%
  dplyr::arrange(p.value) %>%
  dplyr::slice_head(n = 20) %>%
  dplyr::select(
    SNP_ID,
    estimate_season = estimate,

```

```

    p_season = p.value,
    log10p_season = log10p,
    sig_bh_season = sig_bh,
    sig_bonf_season = sig_bonf
  ) %>%
  dplyr::left_join(
    scan_season_pc_adj %>%
      dplyr::filter(trait == trait_to_plot) %>%
      dplyr::select(
        SNP_ID,
        estimate_pc = estimate,
        p_pc = p.value,
        log10p_pc = log10p,
        sig_bh_pc = sig_bh,
        sig_bonf_pc = sig_bonf
      ),
    by = "SNP_ID"
  ) %>%
  dplyr::mutate(
    abs_est_change = abs(estimate_pc - estimate_season),
    p_ratio = p_pc / p_season
  )

knitr::kable(top_markers_compare, digits = 4)

```

SNP_ID	estimate_season	log10p_season	sig_bh_season	sig_bonf_season	estimate_pc	log10p_pc	sig_bh_pc	sig_bonf_pc	abs_est_change	p_ratio		
Marker-4017	-0.5752	0	13.2954	TRUE	TRUE	-0.5657	0	12.6677	TRUE	TRUE	0.0095	4.2432
Marker-4013	0.5776	0	12.9074	TRUE	TRUE	0.5768	0	12.4526	TRUE	TRUE	0.0008	2.8495
Marker-4000	-0.5657	0	12.6262	TRUE	TRUE	-0.5537	0	11.7783	TRUE	TRUE	0.0120	7.0456
Marker-4018	0.5667	0	12.1878	TRUE	TRUE	0.5576	0	11.5993	TRUE	TRUE	0.0091	3.8768
Marker-4019	-0.5362	0	11.5618	TRUE	TRUE	-0.5273	0	10.9341	TRUE	TRUE	0.0089	4.2434
Marker-1568	-1.1927	0	11.3502	TRUE	TRUE	-1.1858	0	9.6323	TRUE	TRUE	0.0068	52.2334
Marker-4009	-0.5370	0	11.2134	TRUE	TRUE	-0.5282	0	10.5852	TRUE	TRUE	0.0089	4.2481
Marker-4001	0.5262	0	11.0914	TRUE	TRUE	0.5161	0	10.4086	TRUE	TRUE	0.0101	4.8167
Marker-4016	0.5179	0	10.7453	TRUE	TRUE	0.5144	0	10.4342	TRUE	TRUE	0.0036	2.0469
Marker-1530	-0.7319	0	10.6274	TRUE	TRUE	-0.7619	0	9.0591	TRUE	TRUE	0.0300	37.0088
Marker-3109	0.9307	0	10.4637	TRUE	TRUE	1.0559	0	12.5542	TRUE	TRUE	0.1252	0.0081
Marker-4028	-0.5743	0	10.3774	TRUE	TRUE	-0.5713	0	10.1097	TRUE	TRUE	0.0030	1.8521
Marker-1556	-0.6988	0	10.1953	TRUE	TRUE	-0.7017	0	8.4912	TRUE	TRUE	0.0029	50.6009

SNP_ID	Estimate	season	log10p	sig_bh	sig_bonf	season	log10p	sig_bh	sig_bonf	pc	est	change
Marker-1550	-0.6678	0	9.9122	TRUE	TRUE	-0.6836	0	8.2208	TRUE	TRUE	0.0158	49.1328
Marker-1540	-0.6704	0	9.8844	TRUE	TRUE	-0.6795	0	8.2066	TRUE	TRUE	0.0091	47.6171
Marker-1542	0.6823	0	9.8092	TRUE	TRUE	0.6999	0	8.1473	TRUE	TRUE	0.0176	45.9095
Marker-754	0.5783	0	9.7476	TRUE	TRUE	0.6134	0	10.5785	TRUE	TRUE	0.0352	0.1476
Marker-1531	-0.6506	0	9.6960	TRUE	TRUE	-0.6500	0	7.9714	TRUE	TRUE	0.0006	53.0467
Marker-1534	-0.6772	0	9.6697	TRUE	TRUE	-0.6873	0	7.9860	TRUE	TRUE	0.0101	48.2746
Marker-1561	0.6772	0	9.6697	TRUE	TRUE	0.6873	0	7.9860	TRUE	TRUE	0.0101	48.2746

```
select_marker_set <- function(scan_df, trait_name, rule = c("bh", "bonf", "top"), top_n = 50) {
  rule <- match.arg(rule)

  df <- scan_df %>%
    dplyr::filter(trait == trait_name) %>%
    dplyr::arrange(p.value)

  if (rule == "bh") {
    hits <- df %>% dplyr::filter(sig_bh) %>% dplyr::pull(SNP_ID)
    return(unique(hits))
  }

  if (rule == "bonf") {
    hits <- df %>% dplyr::filter(sig_bonf) %>% dplyr::pull(SNP_ID)
    return(unique(hits))
  }

  df %>% dplyr::slice_head(n = top_n) %>% dplyr::pull(SNP_ID) %>% unique()
}
```

```
bh_season <- select_marker_set(scan_season_only_adj, trait_to_plot, rule = "bh")
bh_pc <- select_marker_set(scan_season_pc_adj, trait_to_plot, rule = "bh")

overlap_summary_bh <- tibble::tibble(
  trait = trait_to_plot,
  rule = "BH-FDR",
  n_season = length(bh_season),
  n_pc = length(bh_pc),
  n_overlap = length(intersect(bh_season, bh_pc)),
  jaccard = ifelse(
    length(union(bh_season, bh_pc)) == 0,
    NA_real_,
    length(intersect(bh_season, bh_pc)) / length(union(bh_season, bh_pc))
  )
)

knitr::kable(overlap_summary_bh, digits = 3)
```

trait	rule	n_season	n_pc	n_overlap	jaccard
Lint_percentage	BH-FDR	1160	1740	1090	0.602

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

```

bh_season <- select_marker_set(scan_season_only_adj, trait_to_plot, rule = "bh")
bh_pc      <- select_marker_set(scan_season_pc_adj, trait_to_plot, rule = "bh")

overlap_summary_bh <- tibble::tibble(
  trait = trait_to_plot,
  rule = "BH-FDR",
  n_season = length(bh_season),
  n_pc = length(bh_pc),
  n_overlap = length(intersect(bh_season, bh_pc)),
  jaccard = ifelse(
    length(union(bh_season, bh_pc)) == 0,
    NA_real_,
    length(intersect(bh_season, bh_pc)) / length(union(bh_season, bh_pc))
  )
)

knitr::kable(overlap_summary_bh, digits = 3)

```

trait	rule	n_season	n_pc	n_overlap	jaccard
Lint_percentage	BH-FDR	1160	1740	1090	0.602

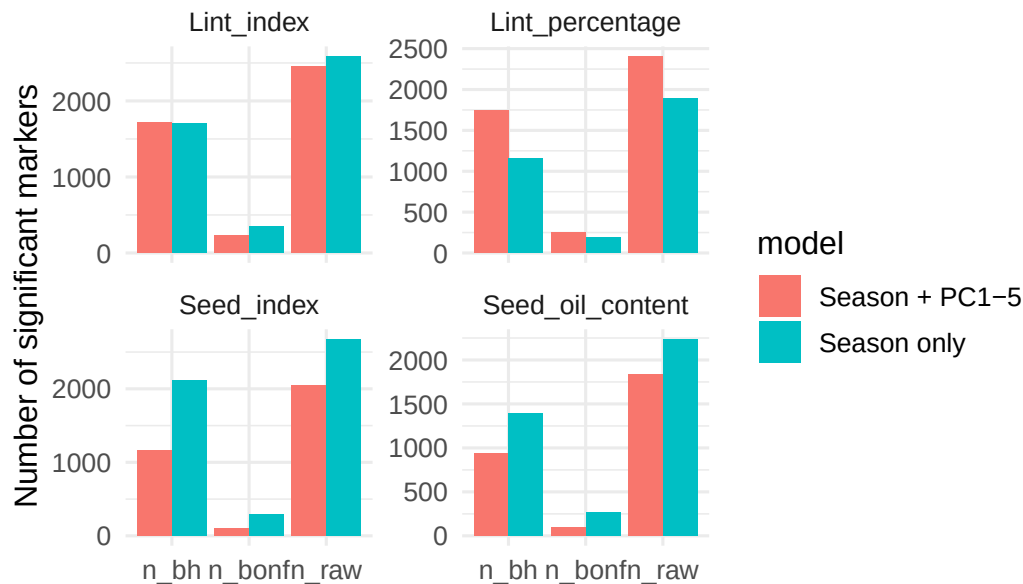
```

sig_summary_long <- sig_summary %>%
  tidyr::pivot_longer(
    cols = c(n_raw, n_bh, n_bonf),
    names_to = "threshold",
    values_to = "n_significant"
  )

ggplot2::ggplot(sig_summary_long, ggplot2::aes(x = threshold, y = n_significant, fill = model)) +
  ggplot2::geom_col(position = "dodge") +
  ggplot2::facet_wrap(~ trait, scales = "free_y") +
  ggplot2::theme_minimal(base_size = 12) +
  ggplot2::labs(
    title = "Number of significant markers across model specifications",
    x = NULL,
    y = "Number of significant markers"
  )

```


Number of significant markers across model specification



Using `Lint_percentage` as the example trait, the report compares the top markers under both model specifications. Many top hits, including markers such as `Marker-4017`, `Marker-4013`, and `Marker-4000`, retain very similar effect sizes across models. At the same time, some markers shift substantially in significance. Under BH, the overlap between the two significant-marker sets is **1090**, with a **Jaccard index of 0.602**.

The interpretation is that **PC adjustment changes the boundary of the significance map more than it overturns the strongest signals outright**. Strong leading hits are moderately robust, but the broader set of candidate markers is model-sensitive. A Jaccard index of 0.602 indicates meaningful overlap, but certainly not equivalence.

Accordingly, the most credible candidate regions are those that remain highly ranked across both model specifications and continue to appear under shrinkage selection.

```
compare_pvals <- scan_season_only_adj %>%
  dplyr::select(trait, SNP_ID, log10p_season = log10p, est_season = estimate) %>%
  dplyr::left_join(
    scan_season_pc_adj %>%
      dplyr::select(trait, SNP_ID, log10p_pc = log10p, est_pc = estimate),
    by = c("trait", "SNP_ID")
  )
```

```
# ---- Compare season-only vs season+PC marker statistics ----
```

```
compare_scan_df <- scan_season_only_adj %>%
  dplyr::select(
    trait, SNP_ID,
    p_season = p.value,
    log10p_season = log10p,
    estimate_season = estimate,
    p_adj_bh_season = p_adj_bh,
    p_adj_bonf_season = p_adj_bonf
  ) %>%
  dplyr::left_join(
    scan_season_pc_adj %>%
```

```

dplyr::select(
  trait, SNP_ID,
  p_pc = p.value,
  log10p_pc = log10p,
  estimate_pc = estimate,
  p_adj_bh_pc = p_adj_bh,
  p_adj_bonf_pc = p_adj_bonf
),
by = c("trait", "SNP_ID")
)

```

```

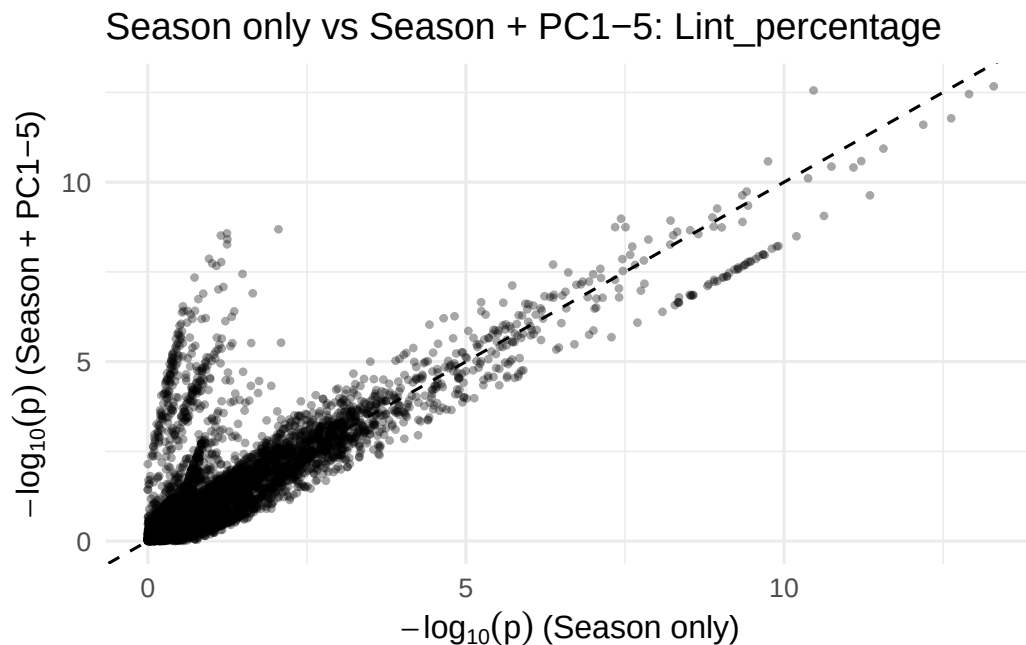
trait_to_plot <- "Lint_percentage"

```

```

ggplot2::ggplot(
  compare_scan_df %>% dplyr::filter(trait == trait_to_plot),
  ggplot2::aes(x = log10p_season, y = log10p_pc)
) +
  ggplot2::geom_point(alpha = 0.35, size = 0.8) +
  ggplot2::geom_abline(slope = 1, intercept = 0, linetype = "dashed") +
  ggplot2::theme_minimal(base_size = 12) +
  ggplot2::labs(
    title = paste("Season only vs Season + PC1-5:", trait_to_plot),
    x = expression(-log[10](p)~"(Season only)"),
    y = expression(-log[10](p)~"(Season + PC1-5)")
  )

```

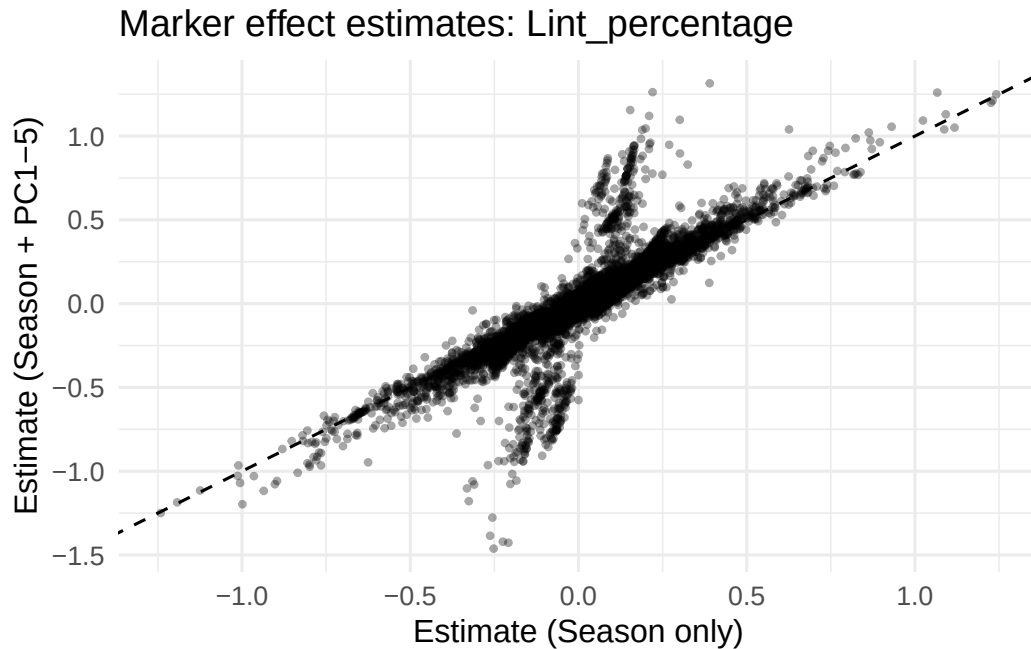


```

ggplot2::ggplot(
  compare_scan_df %>% dplyr::filter(trait == trait_to_plot),
  ggplot2::aes(x = estimate_season, y = estimate_pc)
) +
  ggplot2::geom_point(alpha = 0.35, size = 0.8) +
  ggplot2::geom_abline(slope = 1, intercept = 0, linetype = "dashed") +

```

```
ggplot2::theme_minimal(base_size = 12) +
ggplot2::labs(
  title = paste("Marker effect estimates:", trait_to_plot),
  x = "Estimate (Season only)",
  y = "Estimate (Season + PC1-5)"
)
```



```
scan_diff_summary <- compare_scan_df %>%
  dplyr::group_by(trait) %>%
  dplyr::summarise(
    cor_log10p = cor(log10p_season, log10p_pc, use = "complete.obs"),
    cor_estimate = cor(estimate_season, estimate_pc, use = "complete.obs"),
    mean_abs_est_diff = mean(abs(estimate_pc - estimate_season), na.rm = TRUE),
    n_bh_season = sum(p_adj_bh_season < 0.05, na.rm = TRUE),
    n_bh_pc = sum(p_adj_bh_pc < 0.05, na.rm = TRUE),
    n_bonf_season = sum(p_adj_bonf_season < 0.05, na.rm = TRUE),
    n_bonf_pc = sum(p_adj_bonf_pc < 0.05, na.rm = TRUE),
    .groups = "drop"
  )

knitr::kable(scan_diff_summary, digits = 3)
```

trait	cor_log10p	cor_estimate	mean_abs_est_diff	nh_season	nh_pc	bonf_season	bonf_pc
Lint_index	0.631	0.752	0.042	1701	1711	347	233
Lint_percentage	0.827	0.867	0.079	1160	1740	191	248
Seed_index	0.558	0.695	0.048	2121	1159	286	104
Seed_oil_content	0.623	0.730	0.074	1389	942	273	100

```
trait_to_check <- "Lint_percentage"

top20_markers <- scan_season_only_adj %>%
```

```

dplyr::filter(trait == trait_to_check) %>%
dplyr::arrange(p.value) %>%
dplyr::slice_head(n = 20) %>%
dplyr::pull(SNP_ID)

# ---- Create season-residualised traits ----

make_season_residuals <- function(df, trait_names) {
  df <- as.data.frame(df)

  if (!"Season" %in% names(df)) {
    stop("Column 'Season' not found in analysis_df.")
  }

  for (tr in trait_names) {
    if (!tr %in% names(df)) {
      stop(paste("Trait not found in analysis_df:", tr))
    }

    tmp <- data.frame(
      .y = df[[tr]],
      .season = df[["Season"]]
    )

    keep <- stats::complete.cases(tmp)
    resid_vec <- rep(NA_real_, nrow(df))

    fit <- stats::lm(.y ~ .season, data = tmp[keep, , drop = FALSE])
    resid_vec[keep] <- stats::resid(fit)

    df[[paste0(tr, "_season_resid")]] <- resid_vec
  }

  df
}

analysis_df_resid <- make_season_residuals(analysis_df, trait_names)

cat("Created analysis_df_resid\n")

Created analysis_df_resid
print(dim(analysis_df_resid))

[1] 768 6539

print(colnames(analysis_df_resid)[grepl("_season_resid$", colnames(analysis_df_resid))])

[1] "Lint_percentage_season_resid" "Seed_index_season_resid"
[3] "Lint_index_season_resid"      "Seed_oil_content_season_resid"

# ---- Exact LM on season-residualised phenotype ----

run_exact_resid_lm <- function(df, resid_trait, marker_names) {
  df <- as.data.frame(df)

```

```

if (!resid_trait %in% names(df)) {
  stop(paste("Residual trait not found in df:", resid_trait))
}

out <- lapply(marker_names, function(mk) {
  if (!mk %in% names(df)) {
    stop(paste("Marker not found in df:", mk))
  }

  tmp <- data.frame(
    .y = df[[resid_trait]],
    .marker = df[[mk]]
  )

  tmp <- tmp[stats::complete.cases(tmp), , drop = FALSE]

  if (nrow(tmp) < 3 || stats::var(tmp$.marker) == 0) {
    return(tibble::tibble(
      resid_trait = resid_trait,
      SNP_ID = mk,
      estimate = NA_real_,
      std.error = NA_real_,
      statistic = NA_real_,
      p.value = NA_real_
    ))
  }

  fit <- stats::lm(.y ~ .marker, data = tmp)
  co <- broom::tidy(fit)
  co <- co[co$term == ".marker", , drop = FALSE]

  tibble::tibble(
    resid_trait = resid_trait,
    SNP_ID = mk,
    estimate = co$estimate,
    std.error = co$std.error,
    statistic = co$statistic,
    p.value = co$p.value
  )
})

dplyr::bind_rows(out)
}

exact_resid_lm_results <- run_exact_resid_lm(
  df = analysis_df_resid,
  resid_trait = paste0(trait_to_check, "_season_resid"),
  marker_names = top20_markers
)

knitr::kable(exact_resid_lm_results, digits = 4)

```

resid_trait	SNP_ID	estimate	std.error	statistic	p.value
Lint_percentage_season_resid	Marker-4017	-0.5752	0.0748	-7.6848	0
Lint_percentage_season_resid	Marker-4013	0.5776	0.0764	7.5604	0
Lint_percentage_season_resid	Marker-4000	-0.5657	0.0757	-7.4692	0
Lint_percentage_season_resid	Marker-4018	0.5667	0.0774	7.3252	0
Lint_percentage_season_resid	Marker-4019	-0.5362	0.0754	-7.1153	0
Lint_percentage_season_resid	Marker-1568	-1.1927	0.1693	-7.0431	0
Lint_percentage_season_resid	Marker-4009	-0.5370	0.0768	-6.9962	0
Lint_percentage_season_resid	Marker-4001	0.5262	0.0757	6.9540	0
Lint_percentage_season_resid	Marker-4016	0.5179	0.0758	6.8334	0
Lint_percentage_season_resid	Marker-1530	-0.7319	0.1078	-6.7919	0
Lint_percentage_season_resid	Marker-3109	0.9307	0.1382	6.7339	0
Lint_percentage_season_resid	Marker-4028	-0.5743	0.0857	-6.7031	0
Lint_percentage_season_resid	Marker-1556	-0.6988	0.1053	-6.6379	0
Lint_percentage_season_resid	Marker-1550	-0.6678	0.1022	-6.5353	0
Lint_percentage_season_resid	Marker-1540	-0.6704	0.1027	-6.5251	0
Lint_percentage_season_resid	Marker-1542	0.6823	0.1050	6.4976	0
Lint_percentage_season_resid	Marker-754	0.5783	0.0893	6.4750	0
Lint_percentage_season_resid	Marker-1531	-0.6506	0.1008	-6.4560	0
Lint_percentage_season_resid	Marker-1534	-0.6772	0.1050	-6.4463	0
Lint_percentage_season_resid	Marker-1561	0.6772	0.1050	6.4463	0

```

compare_resid_vs_scan <- scan_season_only_adj %>%
  dplyr::filter(trait == trait_to_check, SNP_ID %in% top20_markers) %>%
  dplyr::select(
    SNP_ID,
    estimate_scan = estimate,
    p_scan = p.value,
    log10p_scan = log10p
  ) %>%
  dplyr::left_join(
    exact_resid_lm_results %>%
      dplyr::select(
        SNP_ID,
        estimate_resid_lm = estimate,
        p_resid_lm = p.value
      ),
    by = "SNP_ID"
  )

knitr::kable(compare_resid_vs_scan, digits = 4)

```

SNP_ID	estimate_scan	p_scan	log10p_scan	estimate_resid_lm	p_resid_lm
Marker-4017	-0.5752	0	13.2954	-0.5752	0
Marker-4013	0.5776	0	12.9074	0.5776	0
Marker-4000	-0.5657	0	12.6262	-0.5657	0
Marker-4018	0.5667	0	12.1878	0.5667	0
Marker-4019	-0.5362	0	11.5618	-0.5362	0
Marker-1568	-1.1927	0	11.3502	-1.1927	0
Marker-4009	-0.5370	0	11.2134	-0.5370	0
Marker-4001	0.5262	0	11.0914	0.5262	0
Marker-4016	0.5179	0	10.7453	0.5179	0

SNP_ID	estimate_scan	p_scan	log10p_scan	estimate_resid_lm	p_resid_lm
Marker-1530	-0.7319	0	10.6274	-0.7319	0
Marker-3109	0.9307	0	10.4637	0.9307	0
Marker-4028	-0.5743	0	10.3774	-0.5743	0
Marker-1556	-0.6988	0	10.1953	-0.6988	0
Marker-1550	-0.6678	0	9.9122	-0.6678	0
Marker-1540	-0.6704	0	9.8844	-0.6704	0
Marker-1542	0.6823	0	9.8092	0.6823	0
Marker-754	0.5783	0	9.7476	0.5783	0
Marker-1531	-0.6506	0	9.6960	-0.6506	0
Marker-1534	-0.6772	0	9.6697	-0.6772	0
Marker-1561	0.6772	0	9.6697	0.6772	0

The report residualises `Lint_percentage` on season and reruns exact linear models for the top 20 markers. The estimates and p-values closely reproduce the results from the fast scan.

This is an **implementation validation**, not a new biological result. It shows that the fast residualised scanning approach is algebraically consistent with exact regression for the season-only setup. That is reassuring for the computation.

But this does not validate the underlying modelling assumptions. It confirms that the code is behaving correctly, not that independence, specification, or uncertainty quantification are fully adequate.

```

trait_to_check <- "Lint_percentage"

top20_markers <- scan_season_only_adj %>%
  dplyr::filter(trait == trait_to_check) %>%
  dplyr::arrange(p.value) %>%
  dplyr::slice_head(n = 20) %>%
  dplyr::pull(SNP_ID)

# ---- Create season-residualised traits ----

make_season_residuals <- function(df, trait_names) {
  df <- as.data.frame(df)

  for (tr in trait_names) {
    stopifnot(tr %in% names(df))

    tmp <- data.frame(
      .y = df[[tr]],
      .season = df[["Season"]]
    )

    keep <- stats::complete.cases(tmp)
    resid_vec <- rep(NA_real_, nrow(df))

    fit <- stats::lm(.y ~ .season, data = tmp[keep, , drop = FALSE])
    resid_vec[keep] <- stats::resid(fit)

    df[[paste0(tr, "_season_resid")]] <- resid_vec
  }

  df

```

```

}

analysis_df_resid <- make_season_residuals(analysis_df, trait_names)

# optional quick check
colnames(analysis_df_resid)[grepl("_season_resid$", colnames(analysis_df_resid))]

[1] "Lint_percentage_season_resid" "Seed_index_season_resid"
[3] "Lint_index_season_resid"      "Seed_oil_content_season_resid"

stopifnot(exists("analysis_df_resid"))
stopifnot(is.data.frame(analysis_df_resid))
stopifnot(all(paste0(trait_names, "_season_resid") %in% names(analysis_df_resid)))

exact_resid_lm_results <- run_exact_resid_lm(
  df = analysis_df_resid,
  resid_trait = paste0(trait_to_check, "_season_resid"),
  marker_names = top20_markers
)

knitr::kable(exact_resid_lm_results, digits = 4)

```

resid_trait	SNP_ID	estimate	std.error	statistic	p.value
Lint_percentage_season_resid	Marker-4017	-0.5752	0.0748	-7.6848	0
Lint_percentage_season_resid	Marker-4013	0.5776	0.0764	7.5604	0
Lint_percentage_season_resid	Marker-4000	-0.5657	0.0757	-7.4692	0
Lint_percentage_season_resid	Marker-4018	0.5667	0.0774	7.3252	0
Lint_percentage_season_resid	Marker-4019	-0.5362	0.0754	-7.1153	0
Lint_percentage_season_resid	Marker-1568	-1.1927	0.1693	-7.0431	0
Lint_percentage_season_resid	Marker-4009	-0.5370	0.0768	-6.9962	0
Lint_percentage_season_resid	Marker-4001	0.5262	0.0757	6.9540	0
Lint_percentage_season_resid	Marker-4016	0.5179	0.0758	6.8334	0
Lint_percentage_season_resid	Marker-1530	-0.7319	0.1078	-6.7919	0
Lint_percentage_season_resid	Marker-3109	0.9307	0.1382	6.7339	0
Lint_percentage_season_resid	Marker-4028	-0.5743	0.0857	-6.7031	0
Lint_percentage_season_resid	Marker-1556	-0.6988	0.1053	-6.6379	0
Lint_percentage_season_resid	Marker-1550	-0.6678	0.1022	-6.5353	0
Lint_percentage_season_resid	Marker-1540	-0.6704	0.1027	-6.5251	0
Lint_percentage_season_resid	Marker-1542	0.6823	0.1050	6.4976	0
Lint_percentage_season_resid	Marker-754	0.5783	0.0893	6.4750	0
Lint_percentage_season_resid	Marker-1531	-0.6506	0.1008	-6.4560	0
Lint_percentage_season_resid	Marker-1534	-0.6772	0.1050	-6.4463	0
Lint_percentage_season_resid	Marker-1561	0.6772	0.1050	6.4463	0

```

compare_resid_vs_scan <- scan_season_only_adj %>%
  dplyr::filter(trait == trait_to_check, SNP_ID %in% top20_markers) %>%
  dplyr::select(
    SNP_ID,
    estimate_scan = estimate,
    p_scan = p.value,
    log10p_scan = log10p
  ) %>%
  dplyr::left_join(

```



```

exact_resid_lm_results %>%
  dplyr::select(
    SNP_ID,
    estimate_resid_lm = estimate,
    p_resid_lm = p.value
  ),
  by = "SNP_ID"
)

```

```
knitr::kable(compare_resid_vs_scan, digits = 4)
```

SNP_ID	estimate_scan	p_scan	log10p_scan	estimate_resid_lm	p_resid_lm
Marker-4017	-0.5752	0	13.2954	-0.5752	0
Marker-4013	0.5776	0	12.9074	0.5776	0
Marker-4000	-0.5657	0	12.6262	-0.5657	0
Marker-4018	0.5667	0	12.1878	0.5667	0
Marker-4019	-0.5362	0	11.5618	-0.5362	0
Marker-1568	-1.1927	0	11.3502	-1.1927	0
Marker-4009	-0.5370	0	11.2134	-0.5370	0
Marker-4001	0.5262	0	11.0914	0.5262	0
Marker-4016	0.5179	0	10.7453	0.5179	0
Marker-1530	-0.7319	0	10.6274	-0.7319	0
Marker-3109	0.9307	0	10.4637	0.9307	0
Marker-4028	-0.5743	0	10.3774	-0.5743	0
Marker-1556	-0.6988	0	10.1953	-0.6988	0
Marker-1550	-0.6678	0	9.9122	-0.6678	0
Marker-1540	-0.6704	0	9.8844	-0.6704	0
Marker-1542	0.6823	0	9.8092	0.6823	0
Marker-754	0.5783	0	9.7476	0.5783	0
Marker-1531	-0.6506	0	9.6960	-0.6506	0
Marker-1534	-0.6772	0	9.6697	-0.6772	0
Marker-1561	0.6772	0	9.6697	0.6772	0

```

run_lasso_trait_checked <- function(df, trait, marker_names,
                                   covariates = c("Season", "PC1", "PC2", "PC3", "PC4", "PC5"),
                                   alpha = 1,
                                   nfolds = 10,
                                   seed = 123) {

  df <- as.data.frame(df)

  stopifnot(trait %in% names(df))
  stopifnot(all(covariates %in% names(df)))
  stopifnot(all(marker_names %in% names(df)))

  keep <- complete.cases(df[, c(trait, covariates), drop = FALSE])
  df <- df[keep, , drop = FALSE]

  y <- df[[trait]]
  X_marker <- prep_marker_matrix(df, marker_names)

  X_cov <- model.matrix(
    stats::reformulate(covariates),

```

```

    data = df
  )[, -1, drop = FALSE]

  x_all <- cbind(X_cov, X_marker)
  cov_names <- colnames(X_cov)

  penalty_factor <- c(rep(0, ncol(X_cov)), rep(1, ncol(X_marker)))

  stopifnot(length(penalty_factor) == ncol(x_all))

  set.seed(seed)
  cvfit <- glmnet::cv.glmnet(
    x = x_all,
    y = y,
    family = "gaussian",
    alpha = alpha,
    nfolds = nfolds,
    standardize = TRUE,
    intercept = TRUE,
    penalty.factor = penalty_factor
  )

  coef_min <- coef(cvfit, s = "lambda.min")
  coef_1se <- coef(cvfit, s = "lambda.1se")

  nz_min <- rownames(coef_min)[as.vector(coef_min != 0)]
  nz_1se <- rownames(coef_1se)[as.vector(coef_1se != 0)]

  nz_min <- setdiff(nz_min, "(Intercept)")
  nz_1se <- setdiff(nz_1se, "(Intercept)")

  covariates_nonzero_min <- intersect(nz_min, cov_names)
  covariates_nonzero_1se <- intersect(nz_1se, cov_names)

  selected_markers_min <- setdiff(nz_min, cov_names)
  selected_markers_1se <- setdiff(nz_1se, cov_names)

  tibble::tibble(
    trait = trait,
    lambda_min = cvfit$lambda.min,
    lambda_1se = cvfit$lambda.1se,
    n_covariates_nonzero_min = length(covariates_nonzero_min),
    n_covariates_nonzero_1se = length(covariates_nonzero_1se),
    n_selected_markers_min = length(selected_markers_min),
    n_selected_markers_1se = length(selected_markers_1se),
    covariates_nonzero_min = list(covariates_nonzero_min),
    covariates_nonzero_1se = list(covariates_nonzero_1se),
    selected_markers_min = list(selected_markers_min),
    selected_markers_1se = list(selected_markers_1se),
    cvfit = list(cvfit)
  )
}

```

```

lasso_results_checked <- lapply(trait_names, function(tr) {
  run_lasso_trait_checked(
    df = analysis_df,
    trait = tr,
    marker_names = marker_names,
    covariates = c("Season", "PC1", "PC2", "PC3", "PC4", "PC5"),
    alpha = 1,
    nfolds = 10,
    seed = 123
  )
}) %>%
  dplyr::bind_rows()

lasso_results_checked %>%
  dplyr::select(
    trait,
    lambda_min, lambda_1se,
    n_covariates_nonzero_min, n_covariates_nonzero_1se,
    n_selected_markers_min, n_selected_markers_1se
  ) %>%
  knitr::kable(digits = 3)

```

trait	lambda_min	lambda_1se	n_covariates_nonzero_min	n_covariates_nonzero_1se	n_selected_markers_min	n_selected_markers_1se
Lint_percentage	0.023	0.062	7	7	240	160
Seed_index	0.006	0.015	7	7	239	171
Lint_index	0.005	0.012	7	7	222	190
Seed_oil_content	0.017	0.040	7	7	212	140

```

lasso_cov_diag <- lasso_results_checked %>%
  dplyr::select(trait, covariates_nonzero_min, covariates_nonzero_1se)

knitr::kable(
  lasso_cov_diag %>%
    dplyr::mutate(
      covariates_nonzero_min = purrr::map_chr(covariates_nonzero_min, ~ paste(.x, collapse = ", ")),
      covariates_nonzero_1se = purrr::map_chr(covariates_nonzero_1se, ~ paste(.x, collapse = ", ")),
    ),
  digits = 3
)

```

trait	covariates_nonzero_min	covariates_nonzero_1se
Lint_percentage	SeasonSeason_2, SeasonSeason_3, PC1, PC2, PC3, PC4, PC5	SeasonSeason_2, SeasonSeason_3, PC1, PC2, PC3, PC4, PC5
Seed_index	SeasonSeason_2, SeasonSeason_3, PC1, PC2, PC3, PC4, PC5	SeasonSeason_2, SeasonSeason_3, PC1, PC2, PC3, PC4, PC5
Lint_index	SeasonSeason_2, SeasonSeason_3, PC1, PC2, PC3, PC4, PC5	SeasonSeason_2, SeasonSeason_3, PC1, PC2, PC3, PC4, PC5
Seed_oil_content	SeasonSeason_2, SeasonSeason_3, PC1, PC2, PC3, PC4, PC5	SeasonSeason_2, SeasonSeason_3, PC1, PC2, PC3, PC4, PC5

```

lasso_selected_min <- lasso_results_checked %>%
  dplyr::select(trait, selected_markers_min) %>%
  tidyr::unnest_longer(selected_markers_min, values_to = "SNP_ID") %>%
  dplyr::filter(!is.na(SNP_ID))

lasso_selected_1se <- lasso_results_checked %>%
  dplyr::select(trait, selected_markers_1se) %>%
  tidyr::unnest_longer(selected_markers_1se, values_to = "SNP_ID") %>%
  dplyr::filter(!is.na(SNP_ID))

lasso_results_final <- lapply(trait_names, function(tr) {
  run_lasso_trait_checked(
    df = analysis_df,
    trait = tr,
    marker_names = marker_names,
    covariates = c("Season"),
    alpha = 1,
    nfolds = 10,
    seed = 123
  )
}) %>%
  dplyr::bind_rows()

lasso_results_final %>%
  dplyr::select(
    trait,
    lambda_min, lambda_1se,
    n_covariates_nonzero_min, n_covariates_nonzero_1se,
    n_selected_markers_min, n_selected_markers_1se
  ) %>%
  knitr::kable(digits = 3)

```

trait	lambda_min	lambda_1se	n_covariates_nonzero_min	n_covariates_nonzero_1se	n_selected_markers_min	n_selected_markers_1se
Lint_percentage	0.024	0.059	2	2	231	165
Seed_index	0.006	0.015	2	2	247	181
Lint_index	0.005	0.014	2	2	220	171
Seed_oil_content	0.018	0.037	2	2	215	161

```

lasso_final_selected_min <- lasso_results_final %>%
  dplyr::select(trait, selected_markers_min) %>%
  tidyr::unnest_longer(selected_markers_min, values_to = "SNP_ID") %>%
  dplyr::filter(!is.na(SNP_ID))

lasso_final_selected_1se <- lasso_results_final %>%
  dplyr::select(trait, selected_markers_1se) %>%
  tidyr::unnest_longer(selected_markers_1se, values_to = "SNP_ID") %>%
  dplyr::filter(!is.na(SNP_ID))

```

The report uses cross-validated `glmnet` with `alpha = 1`, leaving season and PC covariates unpenalised. Under the **Season + PC1–5** specification, `lambda.min` selects roughly **212–240 markers** and `lambda.1se` roughly **140–190 markers**, while all seven covariates remain non-zero. Under the **Season only** specification, `lambda.min` still selects roughly **220–247 markers**, and `lambda.1se` still keeps around **161–181 markers**.

This is highly informative: **even after shrinkage, the selected marker set remains large**. That implies the signal is not extremely sparse. Instead, the architecture looks more like a combination of correlated marker sets, block-level signal, and/or moderately diffuse genetic effects.

There are important interpretive limits, though. Under strong predictor correlation, LASSO often chooses representatives from correlated groups rather than identifying true causal variables. So the selected set is better understood as a **predictive subset** than a causal one. Also, the report does not place LASSO alongside ridge or elastic net, so the multi-marker shrinkage comparison is still incomplete.

```
trait_to_plot <- "Lint_percentage"

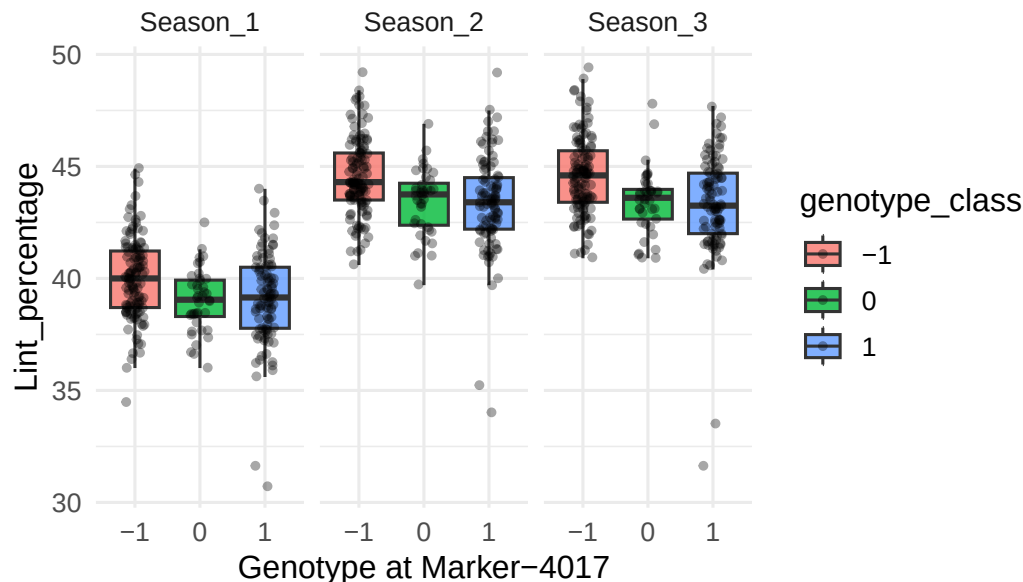
top_snp <- scan_season_pc_adj %>%
  dplyr::filter(trait == trait_to_plot) %>%
  dplyr::arrange(p.value) %>%
  dplyr::slice(1) %>%
  dplyr::pull(SNP_ID)

top_snp

[1] "Marker-4017"

analysis_df %>%
  dplyr::mutate(
    genotype_class = factor(.data[[top_snp]])
  ) %>%
  ggplot(aes(x = genotype_class, y = .data[[trait_to_plot]], fill = genotype_class)) +
  geom_boxplot(outlier.shape = NA, alpha = 0.8) +
  geom_jitter(width = 0.15, alpha = 0.35, size = 1) +
  facet_wrap(~ Season) +
  labs(
    title = paste("Phenotype by genotype class:", top_snp),
    x = paste("Genotype at", top_snp),
    y = trait_to_plot
  ) +
  theme_minimal(base_size = 12)
```

Phenotype by genotype class: Marker-4017



Marker-4017 emerges as the top SNP for `Lint_percentage` under the season+PC model. The boxplots by genotype class show clear trait separation across genotype classes within seasons. After residualising `Lint_percentage` on `Season + PC1-PC5`, the genotype-class differences remain visible in the residual plot.

This is a strong robustness check at the visual level. It indicates that the Marker-4017 signal is not merely an artefact of seasonal shift or broad genotype structure. Even after removing those broad covariate effects, the genotype classes retain clear phenotypic separation. That said, the proper conclusion is still that Marker-4017 is a strong **candidate signal region**, not necessarily a causal variant by itself.

1.6.1

```
trait_to_plot <- "Lint_percentage"

top_snp <- scan_season_pc_adj %>%
  dplyr::filter(trait == trait_to_plot) %>%
  dplyr::arrange(p.value) %>%
  dplyr::slice(1) %>%
  dplyr::pull(SNP_ID)

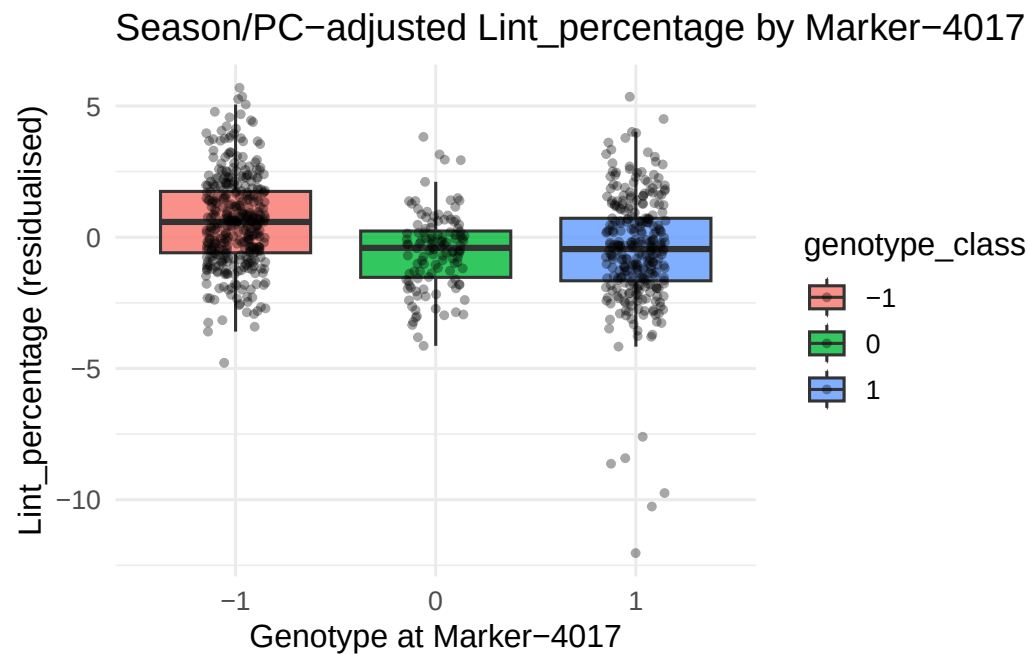
plot_df <- analysis_df %>%
  dplyr::filter(
    !is.na(.data[[trait_to_plot]]),
    !is.na(.data[[top_snp]])
  ) %>%
  dplyr::mutate(
    trait_resid = resid(
      lm(
        as.formula(
          paste(trait_to_plot, "~ Season + PC1 + PC2 + PC3 + PC4 + PC5")
        ),
        data = .
      )
    )
  )
```

```

    ),
    genotype_class = factor(.data[[top_snp]])
  )

ggplot(plot_df, aes(x = genotype_class, y = trait_resid, fill = genotype_class)) +
  geom_boxplot(outlier.shape = NA, alpha = 0.8) +
  geom_jitter(width = 0.15, alpha = 0.35, size = 1) +
  labs(
    title = paste("Season/PC-adjusted", trait_to_plot, "by", top_snp),
    x = paste("Genotype at", top_snp),
    y = paste(trait_to_plot, "(residualised)")
  ) +
  theme_minimal(base_size = 12)

```



```

library(tidyverse)
library(patchwork)

scan_plot_df <- scan_season_pc_adj
# season-only
# scan_plot_df <- scan_season_only_adj

traits_to_plot <- c(
  "Lint_percentage",
  "Seed_index",
  "Lint_index",
  "Seed_oil_content"
)

chr_levels <- unique(map_df$Chromosome)

genome_map <- map_df %>%
  dplyr::filter(!is.na(Chromosome), !is.na(Position_bp)) %>%

```

```

dplyr::mutate(
  Chromosome = factor(Chromosome, levels = chr_levels)
) %>%
dplyr::arrange(Chromosome, Position_bp)

chr_df <- genome_map %>%
  dplyr::group_by(Chromosome) %>%
  dplyr::summarise(chr_len = max(Position_bp), .groups = "drop") %>%
  dplyr::mutate(offset = dplyr::lag(cumsum(chr_len), default = 0))

genome_map2 <- genome_map %>%
  dplyr::left_join(chr_df, by = "Chromosome") %>%
  dplyr::mutate(pos_cum = Position_bp + offset)

axis_df <- genome_map2 %>%
  dplyr::group_by(Chromosome) %>%
  dplyr::summarise(center = mean(range(pos_cum)), .groups = "drop")

plot_manhattan_trait <- function(trait_name, scan_df, genome_map2, axis_df) {

  df_plot <- scan_df %>%
    dplyr::filter(trait == trait_name) %>%
    dplyr::select(SNP_ID, p.value, log10p, Chromosome, Position_bp) %>%
    dplyr::filter(
      !is.na(p.value),
      !is.na(Chromosome),
      !is.na(Position_bp)
    ) %>%
    dplyr::mutate(
      Chromosome = factor(Chromosome, levels = levels(genome_map2$Chromosome))
    ) %>%
    dplyr::left_join(
      genome_map2 %>% dplyr::select(SNP_ID, pos_cum),
      by = "SNP_ID"
    ) %>%
    dplyr::arrange(Chromosome, Position_bp)

  # Bonferroni line
  bonf_line <- -log10(0.05 / nrow(df_plot))

  ggplot(df_plot, aes(x = pos_cum, y = log10p, colour = Chromosome)) +
    geom_point(alpha = 0.75, size = 0.75) +
    geom_hline(
      yintercept = bonf_line,
      linetype = "dashed"
    ) +
    scale_x_continuous(
      breaks = axis_df$center,
      labels = axis_df$Chromosome
    ) +
    labs(
      title = paste("Manhattan plot:", trait_name),
      x = "Chromosome",

```



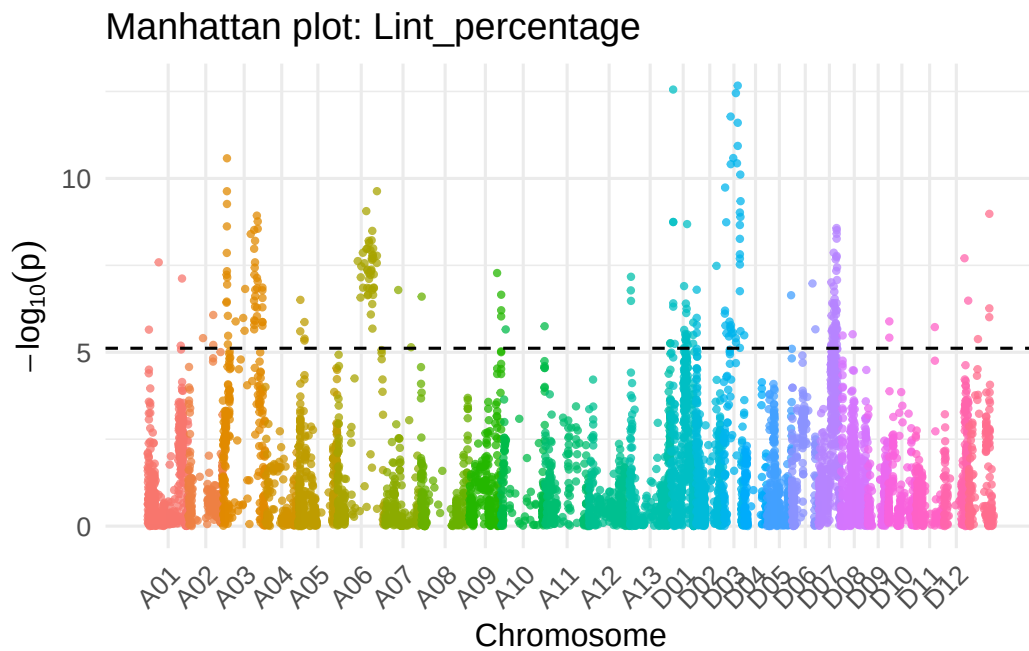
```

    y = expression(-log[10](p))
  ) +
  theme_minimal(base_size = 12) +
  theme(
    legend.position = "none",
    panel.grid.minor.x = element_blank(),
    axis.text.x = element_text(angle = 45, hjust = 1)
  )
}

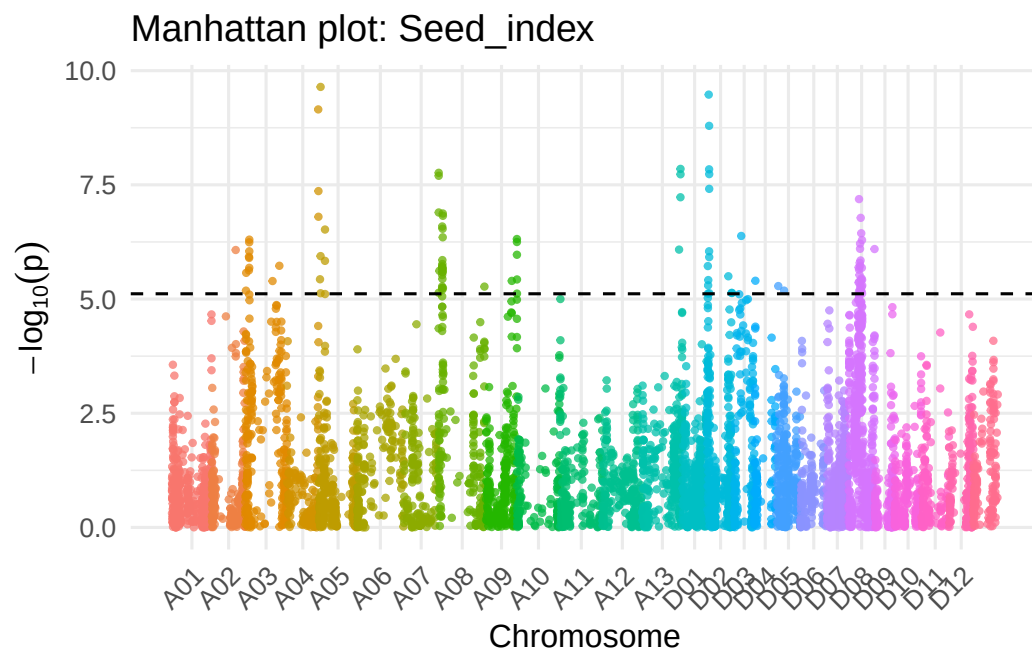
p1 <- plot_manhattan_trait("Lint_percentage", scan_plot_df, genome_map2, axis_df)
p2 <- plot_manhattan_trait("Seed_index", scan_plot_df, genome_map2, axis_df)
p3 <- plot_manhattan_trait("Lint_index", scan_plot_df, genome_map2, axis_df)
p4 <- plot_manhattan_trait("Seed_oil_content", scan_plot_df, genome_map2, axis_df)

p1

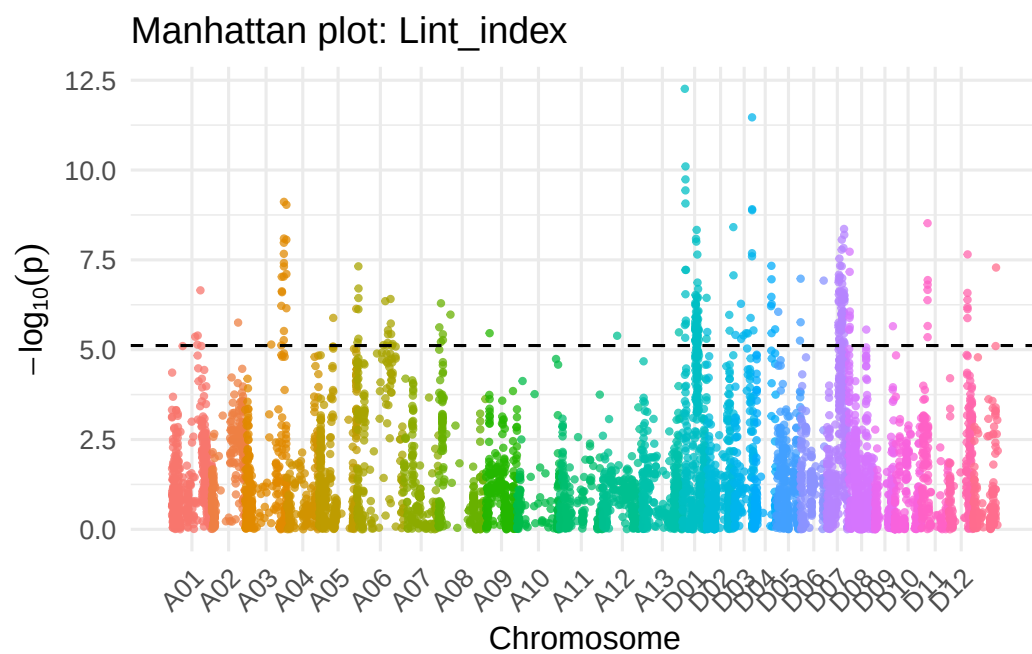
```



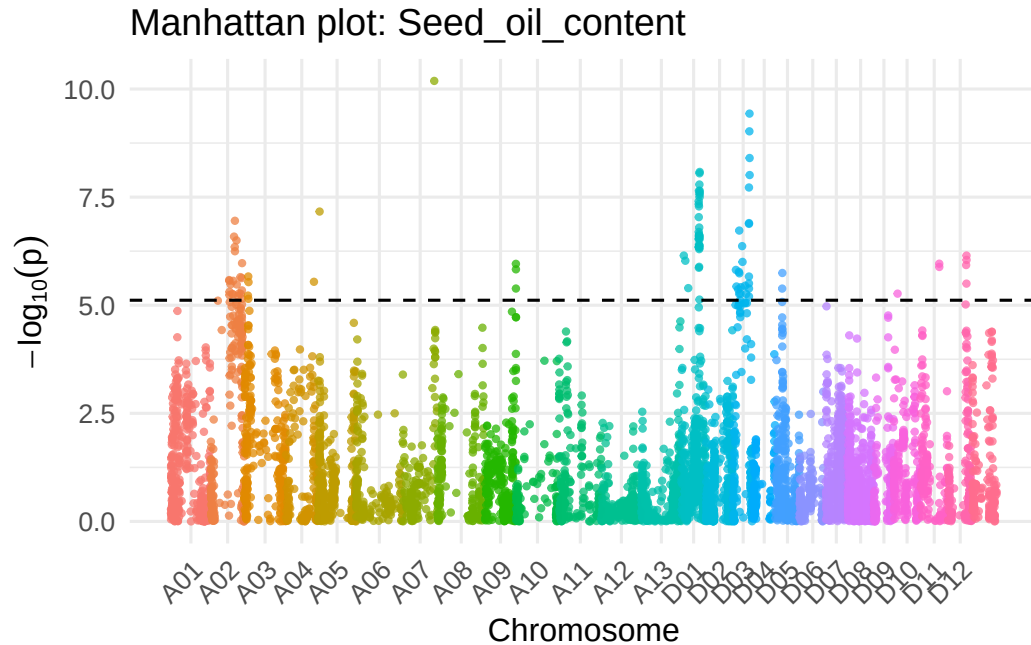
p2



p3



p4



The Manhattan plots display genome-wide significance profiles for `Lint_percentage`, `Seed_index`, `Lint_index`, and `Seed_oil_content`, with Bonferroni thresholds drawn as dashed lines. Across traits, the plots show **multiple elevated regions rather than isolated single spikes**. The peaks for `Lint_percentage` and `Lint_index` appear especially prominent, while `Seed_index` and `Seed_oil_content` also show multiple threshold-exceeding regions.

These plots are fully consistent with the earlier count summaries: the signal landscape is **broad and clustered**, not sparse and isolated. That pattern strongly suggests LD-driven blocks and/or polygenic structure rather than a tiny number of independent loci.

This also explains why LASSO continues to retain well over a hundred markers: the Manhattan plots themselves show that the genome-wide signal pattern is regionally distributed rather than point-like.

1.6.2